
Series Elastic Actuator (SEA) Cable Model

Cable-Driven 2R Planar Manipulator — PyDrake Implementation

Isaac Sim Robotics Project

April 2026

Key Results at a Glance

- **New physics layer:** Series elastic spring between joint-2 big pulley and link-2 anchor
- **Antagonistic cables:** Both green and red cables can transmit force (push + pull)
- **Physically motivated:** Models steel cable elasticity + explicit compliance element
- **Tunable compliance:** k_s from 20 N/m (very soft) to 5000+ N/m (near-rigid)
- **Two motor modes:** **Torque mode** (2nd-order rotor dynamics, default) and **Position mode** (1st-order servo, legacy). Switchable via `--sea-mode`
- **Pluggable motor dynamics:** MotorDynamics ABC with `TorqueMotor` and `PositionServoMotor` subclasses (`actuators/motor_dynamics.py`)
- **Dynamic visualization:** Meshcat spring coil stretches in real-time with δ
- **Comparison mode:** `--compare` overlays compliant vs. rigid on the same plot

Contents

1 Motivation: Why Add Cable Compliance?

The **Computed-Torque (CT) controller** described in the companion document (`ComputedTorquePyDrake`) treats the cable as a **rigid, inextensible tendon**. Joint torque τ_2 is applied instantaneously—there is no delay, no elasticity, and no cable dynamics.

In the real robot, the cable path is:

Motor drum $\rightarrow$ **Steel cable** $\rightarrow$ **Big Pulley** $\rightarrow$ **SPRING** $\rightarrow$ **Link 2 anchor**

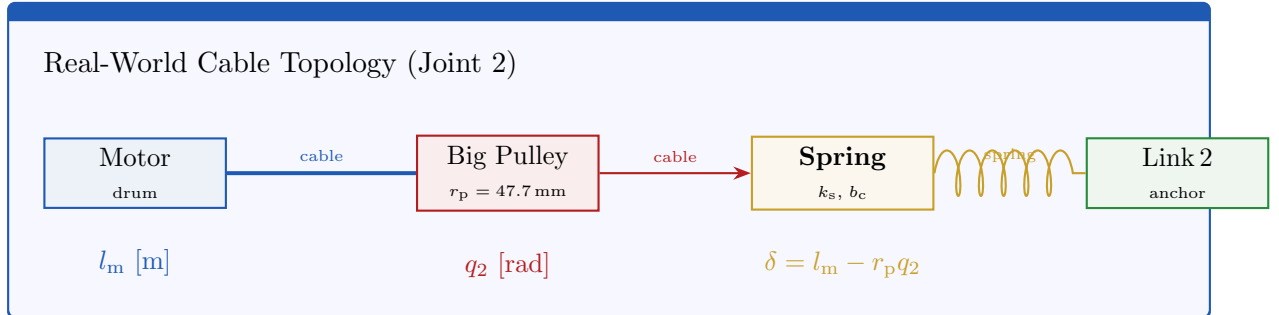
The spring (which may be an explicit compliance element or the intrinsic belt/cable elasticity) introduces:

1. **Torque lag**: The joint torque depends on spring extension, not directly on the motor command.
2. **Energy storage**: The spring stores elastic energy, causing oscillations if underdamped.
3. **Motor–joint decoupling**: The motor and joint move at different rates; the spring absorbs the difference.

This document describes the `script_cup_manipulator_pendulum_tendon_with_spring_sea_pydrake.py` implementation, which adds a **Series Elastic Actuator (SEA)** model to joint 2 while keeping joint 1 as rigid direct-drive.

2 Physical Model

2.1 Physical Cable Routing



2.2 State Variables

The SEA model adds **one new state variable** to the system:

Table 1: SEA state variable (joint 2 cable).

Symbol	Description	Units	Drake storage
l_m	Motor-side cable displacement (paid out by drum)	m	Discrete state

The spring extension is a *derived* quantity:

$$\delta = l_m - r_p q_2 \quad (1)$$

where r_p is the big pulley radius and q_2 is the joint angle. Positive δ means the spring is stretched (motor has paid out more cable than the joint has consumed).

2.3 Spring–Damper Force

The cable force is a linear spring–damper:

$$F_{\text{raw}} = k_s \delta + b_c \dot{\delta} \quad (2)$$

where:

$$\dot{\delta} = \dot{l}_m - r_p \dot{q}_2 \quad (3)$$

Here $\dot{l}_m$ is the motor-side cable velocity, whose computation depends on the motor mode (see Section ??).

2.4 Antagonistic Cable Decomposition

The joint has **two cables** routed in opposite directions around the big pulley:

- **Green cable** (retracting): wraps *Drive* → *IdlerR* → *BigPulley* → *EndL*
- **Red cable** (extending): wraps *Drive* → *IdlerL* → *BigPulley* → *EndR*

Cables can only *pull* (tension $T \geq 0$). The net cable force F_{raw} is decomposed:

$$T_{\text{green}} = \max(F_{\text{raw}}, 0), \quad T_{\text{red}} = \max(-F_{\text{raw}}, 0) \quad (4)$$

The **net signed cable force** applied to the joint is:

$$F_{\text{cable}} = T_{\text{green}} - T_{\text{red}} = F_{\text{raw}} \quad (5)$$

Antagonistic means bidirectional. Unlike a single-cable system (which can only pull), the two-cable antagonistic arrangement allows the cable drive to produce *both positive and negative* joint torque. When $F_{\text{raw}} > 0$ the green cable is taut and the red is slack; when $F_{\text{raw}} < 0$ the red cable is taut and the green is slack. Only one cable carries load at any instant (no pre-tension model).

2.5 Joint Torque

The cable force maps to joint-2 torque through the pulley radius:

$$\tau_2 = r_p \cdot F_{\text{cable}} \quad (6)$$

Joint 1 receives its torque directly from the CT controller (rigid direct-drive):

$$\tau_1 = \tau_{1,\text{des}} \quad (7)$$

3 Motor Model

The motor model is **pluggable**: two modes are supported, selectable at runtime via `--sea-mode`. Both share the same cable spring–damper physics; only the motor dynamics differ.

3.1 Torque Mode (Default — 2nd-Order Rotor Dynamics)

In torque mode the motor runs in MIT torque/impedance mode (the default for quasi-direct-drive actuators like CubeMars AK60-6). The rotor is modelled as an inertia J_m with viscous damping b_m , driven by the commanded torque and loaded by the spring reaction through the gearbox:

$$\boxed{J_m \ddot{\theta}_m = \frac{\tau_{2,\text{des}}}{N} - b_m \dot{\theta}_m - \frac{r_p \cdot F}{N}} \quad (8)$$

where θ_m is the motor-side angle, N is the gear ratio, and F is the cable spring force in Newtons.

Spring extension in linear coordinates:

$$\delta = r_p \left(\frac{\theta_m}{N} - q_2 \right), \quad \dot{\delta} = r_p \left(\frac{\dot{\theta}_m}{N} - \dot{q}_2 \right) \quad (9)$$

Cable force:

$$F = k_s \delta + b_c \dot{\delta} \quad (10)$$

Motor-side resonance frequency (effective torsional spring at the rotor):

$$\omega_n = \sqrt{\frac{r_p^2 k_s}{N^2 J_m}} \quad (11)$$

For AK60-6 with $k_s = 300 \text{ N/m}$, $r_p = 47.75 \text{ mm}$, $N = 6$: $\omega_n \approx 24 \text{ rad/s}$ (4 Hz).

Discrete integration (semi-implicit Euler, velocity first):

$$\dot{\theta}_m[k+1] = \dot{\theta}_m[k] + \Delta t \cdot \ddot{\theta}_m[k] \quad (12)$$

$$\theta_m[k+1] = \theta_m[k] + \Delta t \cdot \dot{\theta}_m[k+1] \quad (13)$$

State: $[\theta_m, \dot{\theta}_m]$ (2 scalars, motor-side).

► Equations implemented by the above code

The motor equation derives from Newton's second law on the rotor:

$$\underbrace{J_m \ddot{\theta}_m}_{\text{rotor inertia}} = \underbrace{\frac{\tau_{2,\text{des}}}{N}}_{\text{motor-side command}} - \underbrace{b_m \dot{\theta}_m}_{\text{viscous friction}} - \underbrace{\frac{r_p \cdot F}{N}}_{\text{spring load reflected to motor}}$$

The spring load is computed in the *linear* domain (δ in metres) to keep k_s in N/m consistent across modes.

3.2 Position Mode (Legacy — 1st-Order Servo)

When the motor runs a factory position controller (CAN position commands), it is modelled as a **first-order low-pass filter**:

$$\boxed{\dot{l}_m = \omega_b (l_m^{\text{des}} - l_m)} \quad (14)$$

where ω_b is the position-servo bandwidth. Discretized via forward Euler:

$$l_m[k+1] = l_m[k] + \Delta t \cdot \omega_b (l_m^{\text{des}}[k] - l_m[k]) \quad (15)$$

The desired motor cable position is computed by **steady-state spring inversion**:

$$l_m^{\text{des}} = r_p q_2 + \frac{\tau_{2,\text{des}}}{k_s r_p} \quad (16)$$

State: $[l_m]$ (1 scalar, cable displacement in metres).

Position-servo bandwidth is $\omega_b = 1/\tau_m$, NOT the no-load joint speed.

The mechanical time constant $\tau_m = 2.5\text{ms}$ gives an open-loop upper bound of $\omega_b = 1/\tau_m \approx 400\text{rad/s}$. However, the closed-loop position-servo bandwidth is $2\text{--}5\times$ lower (depends on the motor's internal PID tuning, which CubeMars does not publish). The default is a conservative $\omega_b = 100\text{rad/s}$.

The no-load joint speed ($\omega_{\text{NL,joint}} = 11.17\text{rad/s}$) limits *maximum cable velocity*, not servo bandwidth. These are different quantities.

3.3 Effect of Parameters

Table 2: Parameter effects on SEA behaviour.

Parameter	Range	Effect
k_s high (5000+ N/m)	stiff	Small $\delta_{ss} \rightarrow$ fast torque $\rightarrow$ near-rigid
k_s low (20–50 N/m)	soft	Large $\delta_{ss} \rightarrow$ slow torque $\rightarrow$ high lag
ω_b high (100+ rad/s)	fast motor (pos. mode)	Tracks l_m^{des} quickly $\rightarrow$ less lag
ω_b low (5–10 rad/s)	slow motor (pos. mode)	Can't keep up $\rightarrow$ more lag at high speed
b_c high	heavy damping	Suppresses oscillation, adds velocity-dependent drag
b_c low	light damping	Allows spring oscillation at natural frequency

The SEA introduces a **fundamental bandwidth limit**: even with perfect CT, the closed-loop cannot be faster than the cable spring natural frequency:

$$\omega_{\text{spring}} = \sqrt{\frac{k_s r_p^2}{I_2}} \quad (17)$$

where I_2 is the reflected inertia at joint 2.

4 CubeMars AK60-6 KV80 Motor

The elbow (joint 2) is driven by a **CubeMars AK60-6 V3.0 KV80** quasi-direct-drive actuator, which integrates a brushless DC motor with a 6:1 planetary gearbox in a compact package. This section explains every datasheet parameter, what it physically means, and how it affects the manipulator's motion.

4.1 Datasheet Specifications

Table 3: CubeMars AK60-6 KV80 — full specification table. All values are *joint side* (after gearbox) unless marked “motor side.”

Parameter	Symbol	Value	Unit
<i>Mechanical</i>			
Gear ratio	N	6	—
Peak torque (joint)	τ_{peak}	9.0	Nm
Continuous torque (joint)	τ_{cont}	3.0	Nm
No-load speed (motor side)	$\omega_{\text{NL,motor}}$	640	rpm
No-load speed (joint side)	$\omega_{\text{NL,joint}}$	11.17	rad/s
Rated speed (motor / joint)		400 / 66.7	rpm
Rotor inertia (motor side)	$I_{\text{r,motor}}$	3.3191×10^{-5}	kg·m ²
Reflected inertia (joint side)	$I_{\text{r,joint}}$	1.195×10^{-3}	kg·m ²
Viscous damping (joint)	b_v	0.12	Nm·s/rad
Back-drive torque	τ_{bd}	0.2	Nm
Mass	m_{motor}	0.460	kg
<i>Electrical</i>			
Torque constant (motor side)	K_T	0.11937	Nm/A
Velocity constant	K_V	80	rpm/V
Rated voltage	V_{bus}	24/48	V
Rated current	I_{rated}	4.0	A DC
Peak current	I_{peak}	11.2	A DC
Pole pairs	p	14	—
Phase resistance	R	0.595	Ω
Phase inductance	L	675	μH
Motor constant	K_m	0.1541	Nm/ $\sqrt{\text{W}}$
Mechanical time constant	τ_m	2.5	ms
Electrical time constant	$\tau_e = L/R$	1.13	ms

4.2 What Each Parameter Means (Plain Language)

Gear ratio ($N = 6$)

The motor shaft turns 6 times for every single turn of the joint. This multiplies torque by 6 but divides speed by 6. A lower gear ratio makes the joint feel “lighter” and more backdrivable; a higher ratio provides more torque but adds friction and inertia.

Effect on manipulator: With $N = 6$, the motor can push with up to 9 Nm at the joint, which is enough for a small planar arm. If we needed more force (e.g., lifting heavy objects), we would pick a higher gear ratio, but the arm would move slower.

Peak torque ($\tau_{\text{peak}} = 9.0 \text{ Nm}$)

The *maximum instantaneous* torque the motor can exert at the joint. This is only available for short bursts (milliseconds to a few seconds) before the motor overheats.

Effect on manipulator: Sets the hard limit on how quickly the arm can accelerate or decelerate. The CT controller clips torque commands to $\pm\tau_{\text{peak}}$. If the trajectory demands more torque than 9 Nm, the arm will track poorly.

Continuous torque ($\tau_{\text{cont}} = 3.0 \text{ Nm}$)

The torque the motor can sustain *indefinitely* without overheating. Think of peak torque as a sprint and continuous torque as a marathon pace.

Effect on manipulator: During steady-state trajectory tracking (constant speed on a rect-angle), the required torque must stay below 3 Nm or the motor will thermally shut down over time. Gravity compensation for the arm typically requires 1–2 Nm, well within this limit.

No-load joint speed ($\omega_{\text{NL,joint}} = 11.17 \text{ rad/s} \approx 107 \text{ rpm}$)

The fastest the joint can spin when there is zero torque load. Derived from the motor’s 640 rpm no-load speed divided by the gear ratio:

$$\omega_{\text{NL,joint}} = \frac{640 \text{ rpm}}{6} \times \frac{2\pi}{60} = 11.17 \text{ rad/s} \quad (18)$$

Effect on manipulator: Limits how fast the end-effector can sweep. This is a *velocity* limit, not a servo bandwidth. In simulation, the motor dynamics mode determines how this is used: torque mode derives motor acceleration from rotor inertia; position mode uses $\omega_b = 1/\tau_m$ as the servo bandwidth (not $\omega_{\text{NL,joint}}$).

Reflected rotor inertia ($I_{\text{r,joint}} = 1.20 \times 10^{-3} \text{ kg}\cdot\text{m}^2$)

How much the spinning motor rotor “feels like” at the joint, amplified by the square of the gear ratio:

$$I_{\text{r,joint}} = N^2 \cdot I_{\text{r,motor}} = 36 \times 3.3191 \times 10^{-5} = 1.195 \times 10^{-3} \text{ kg}\cdot\text{m}^2 \quad (19)$$

Effect on manipulator: Adds to the link-2 inertia that the CT controller must cancel. For our small arm (link-2 inertia $\approx 10^{-3} \text{ kg}\cdot\text{m}^2$), the reflected inertia is *larger* than the link itself, so the arm feels noticeably heavier than its mass alone would suggest.

Viscous damping ($b_v = 0.12 \text{ Nm}\cdot\text{s/rad}$)

A friction torque proportional to joint speed: $\tau_{\text{friction}} = b_v \cdot \dot{q}_2$. Comes from the gearbox, bearings, and back-EMF effects.

Effect on manipulator: Acts as a natural velocity damper. At the typical trajectory speed of $\dot{q}_2 \approx 1 \text{ rad/s}$, this contributes only 0.12 Nm of drag—small compared to the continuous torque limit, so it does not significantly limit motion.

Back-drive torque ($\tau_{\text{bd}} = 0.2 \text{ Nm}$)

The minimum external torque needed to push the joint when the motor is off. Represents static friction (Coulomb friction + cogging torque).

Effect on manipulator: A low back-drive torque (0.2 Nm for this motor) means the arm is **highly backdrivable**—you can easily push it with your finger. This is desirable for safe human-robot interaction and for force sensing without dedicated sensors. In simulation, this parameter is captured by Drake’s joint damping.

Torque constant ($K_T = 0.11937 \text{ Nm/A}$)

Maps electric current (amps) to motor-shaft torque. Higher K_T means more torque per amp, but lower speed per volt.

Effect on manipulator: Determines how much current the motor driver must supply. At peak joint torque: $I_{\text{peak}} = \tau_{\text{peak}}/(N \cdot K_T) = 9/(6 \times 0.11937) = 12.6 \text{ A}$. The datasheet reports 11.2 A peak (driver-limited), so the motor cannot quite reach full 9 Nm in practice.

Rated voltage ($V_{\text{bus}} = 48 \text{ V}$)

The supply voltage that determines maximum speed. Higher voltage $\rightarrow$ higher no-load speed.

Effect on manipulator: At 48 V, the motor achieves its rated 640 rpm. Running at 24 V would halve the max speed but keep torque the same (current-limited). In simulation, voltage is not modelled; motor bandwidth emerges from rotor inertia (torque mode) or position-servo bandwidth ω_b (position mode).

4.3 Mapping Motor Specs to Simulation Parameters

The simulation does *not* model the full electrical dynamics (current loop, PWM, etc.). Instead, the motor’s complex electromechanical behaviour is captured by a few parameters whose mapping depends on the motor mode:

Table 4: Datasheet $\rightarrow$ simulation parameter mapping.

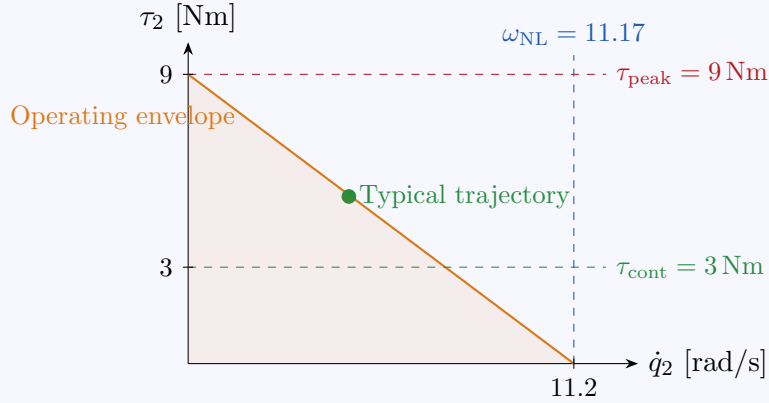
Sim. Parameter	Derived From	Formula / Value	Mode
J_m [kg·m ²] — rotor inertia	Datasheet (motor side)	3.3191×10^{-5}	Torque
b_m [Nm·s/rad] — motor damping	$b_{v,joint}/N^2$	$0.12/36 = 3.33 \times 10^{-3}$	Torque
N — gear ratio	Gearbox spec	6	Both
ω_b [rad/s] — servo bandwidth	$1/\tau_m$ (open-loop)	$1/0.0025 = 400$ (default: 100)	Position
τ_{max} [Nm] — torque clip	Peak torque	$\tau_{peak} = 9.0$	Both
Joint damping [Nm·s/rad]	Viscous damping	$b_v = 0.12$	Both

Position-mode bandwidth: $\omega_b = 1/\tau_m$, **not** $\omega_{NL,joint}$.

The mechanical time constant $\tau_m = J_m \cdot R/K_T^2 = 2.5$ ms gives an open-loop upper bound $\omega_b = 400$ rad/s. The no-load joint speed $\omega_{NL,joint} = 11.17$ rad/s is a *velocity* limit, not a bandwidth. CubeMars does not publish closed-loop position-servo bandwidth, so the CLI default is a conservative 100 rad/s ($\approx 4\times$ below the open-loop limit).

In **torque mode** (default), there is no explicit ω_b ; motor bandwidth emerges from the rotor inertia, damping, and spring dynamics (see Section ??).

Motor Torque-Speed Tradeoff



The motor can produce high torque at low speed or high speed at low torque, but not both simultaneously. Trajectory tracking typically operates in the green region.

5 SEA Equations: Derivation from First Principles

This section derives the complete SEA cable model step by step, starting from the physical setup and ending with the equations implemented in code.

5.1 Step 1: Define the Spring Extension

Consider the cable path: the motor drum pays out a cable of length l_m [m]. This cable wraps around the big pulley (radius r_p) and connects to a spring anchored on link 2. The amount of cable “consumed” by the joint rotation is $r_p \cdot q_2$.

The **spring extension** is simply the difference between what the motor has paid out and what the joint has consumed:

$$\delta = l_m - r_p q_2 \quad (20)$$

- $\delta > 0$: motor has paid out more cable than needed $\rightarrow$ spring is stretched $\rightarrow$ it pulls the joint forward
- $\delta < 0$: joint has rotated further than the motor $\rightarrow$ spring is compressed $\rightarrow$ it pushes back
- $\delta = 0$: spring at rest, no force

5.2 Step 2: Spring–Damper Force

The cable element between the pulley and link 2 behaves as a **parallel spring–damper**:

$$F_{\text{raw}} = \underbrace{k_s \delta}_{\text{elastic}} + \underbrace{b_c \dot{\delta}}_{\text{viscous}} \quad (21)$$

The spring stiffness k_s [N/m] is the cable/belt elasticity (or an explicit compliance element). The damping b_c [N·s/m] models internal friction in the cable.

The spring extension rate $\dot{\delta}$ is:

$$\dot{\delta} = \dot{l}_m - r_p \dot{q}_2 \quad (22)$$

That is, the spring stretches when the motor pulls cable in faster than the joint rotates, and compresses when the joint outruns the motor.

5.3 Step 3: Motor Dynamics

The motor does not jump instantly to the desired state. The dynamics depend on the selected mode:

Torque mode (default). The rotor is an inertia loaded by the spring:

$$J_m \ddot{\theta}_m = \frac{\tau_{2,\text{des}}}{N} - b_m \dot{\theta}_m - \frac{r_p \cdot F}{N} \quad (23)$$

The motor’s bandwidth is *not* a single number—it emerges from the coupled motor–spring dynamics. The motor-side resonance is $\omega_n = \sqrt{r_p^2 k_s / (N^2 J_m)}$, which for AK60-6 and $k_s = 300$ N/m is ≈ 24 rad/s.

Position mode (legacy). The motor tracks a target cable position as a first-order lag:

$$\dot{l}_m = \omega_b (l_m^{\text{des}} - l_m) \quad (24)$$

Where does ω_b come from? It is the position-servo bandwidth, derived from the mechanical time constant: $\omega_b = 1/\tau_m \approx 400$ rad/s (open-loop upper bound for AK60-6). The CLI default is a conservative 100 rad/s because the closed-loop bandwidth is lower (see Section ??).

In discrete time (simulation timestep $\Delta t = 2$ ms):

$$l_m[k+1] = l_m[k] + \Delta t \cdot \omega_b \cdot (l_m^{\text{des}}[k] - l_m[k]) \quad (25)$$

► Equations implemented by the above code

Transfer function of the position-mode motor servo (Laplace domain):

$$\frac{L_m(s)}{L_m^{\text{des}}(s)} = \frac{\omega_b}{s + \omega_b}$$

This is a first-order low-pass with cutoff frequency ω_b . At frequencies above ω_b , the motor cannot track the commanded cable position—this is the fundamental source of lag in position-mode SEA.

5.4 Step 4: Steady-State Spring Inversion

The controller needs to tell the motor *where* to position the cable so that the spring produces the desired torque. At steady state (motor has reached its target, spring extension rate is zero):

$$\tau_{2,\text{des}} = r_p \cdot F_{\text{cable}} = r_p \cdot k_s \cdot \delta_{\text{ss}} \quad (26)$$

$$\Rightarrow \delta_{\text{ss}} = \frac{\tau_{2,\text{des}}}{k_s r_p} \quad (27)$$

$$\Rightarrow l_m^{\text{des}} = \underbrace{r_p q_2}_{\text{joint position}} + \underbrace{\frac{\tau_{2,\text{des}}}{k_s r_p}}_{\text{spring stretch needed}} \quad (28)$$

$$\boxed{l_m^{\text{des}} = r_p q_2 + \frac{\tau_{2,\text{des}}}{k_s r_p}} \quad (29)$$

Plain language: “The motor must pay out enough cable to match the joint angle, *plus* a little extra to stretch the spring by the amount needed to produce the desired force.”

5.5 Step 5: From Cable Force to Joint Torque

The cable force maps to joint torque through the pulley radius (moment arm):

$$\boxed{\tau_2 = r_p \cdot F_{\text{cable}}} \quad (30)$$

With the antagonistic cable arrangement (Section ??), the net cable force is:

$$F_{\text{cable}} = T_{\text{green}} - T_{\text{red}} = F_{\text{raw}}$$

where each tension is non-negative (cables only pull).

5.6 Step 6: Effect of Spring Stiffness on Tracking

The effect of k_s on tracking quality depends on the motor mode. The two modes exhibit *opposite* trends.

Torque mode (default) — stiffer is better. In torque mode the motor bandwidth is *not* fixed; it emerges from the coupled rotor–spring dynamics. The motor-side resonance frequency scales with $\sqrt{k_s}$:

$$\omega_n = \sqrt{\frac{r_p^2 k_s}{N^2 J_m}} \quad \Rightarrow \quad \omega_n \propto \sqrt{k_s} \quad (31)$$

Increasing k_s raises ω_n , making the motor–spring system respond *faster*. Simultaneously, the steady-state spring deflection needed to produce a given torque *decreases*:

$$\delta_{ss} = \frac{\tau_{2,des}}{k_s r_p} \xrightarrow{k_s \uparrow} 0 \quad (32)$$

Both effects work in the same direction: **stiffer springs yield better tracking in torque mode**. In the limit $k_s \rightarrow \infty$, the SEA degenerates to a rigid tendon. This is confirmed by simulation (Section ??): $k_s = 30,000$ N/m achieves near-rigid performance (< 5 mm EE error, τ RMS error 0.004 Nm), while $k_s = 30$ N/m produces ~ 25 mm EE error with $10\times$ worse torque tracking.

Position mode (legacy) — stiffer is worse. In position mode the motor servo has a *fixed* bandwidth ω_b , so the motor position error ε is approximately constant regardless of k_s . The resulting **torque error** scales linearly with stiffness:

$$\Delta\tau_{pos} \approx r_p \cdot k_s \cdot \varepsilon \quad (33)$$

With a soft spring ($k_s = 30$ N/m): $\Delta\tau \approx 0.048 \times 30 \times \varepsilon \approx 1.4 \varepsilon$ Nm.

With a stiff spring ($k_s = 3000$ N/m): $\Delta\tau \approx 0.048 \times 3000 \times \varepsilon \approx 144 \varepsilon$ Nm.

The stiff spring amplifies the same small motor position lag into a *much larger* torque error. The fix: increase ω_b proportionally to k_s .

Torque mode (default): Stiffer spring $\rightarrow$ higher motor–spring resonance $\omega_n \propto \sqrt{k_s} \rightarrow$ **better tracking**. No bandwidth tuning required—the 2nd-order rotor dynamics adapt naturally to stiffer springs.

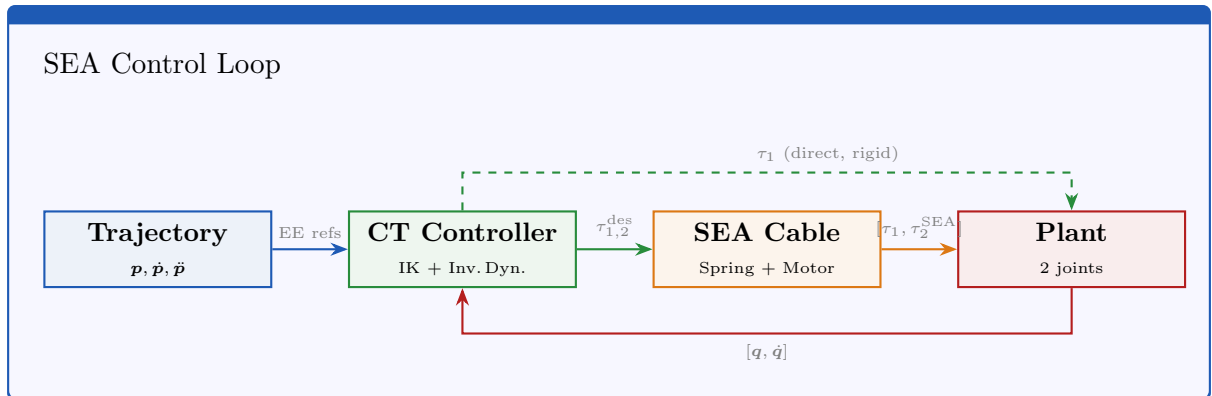
Position mode (legacy): Stiffer spring $\rightarrow$ larger torque error $\Delta\tau \propto k_s \cdot \varepsilon$ if ω_b is held constant. To maintain tracking accuracy:

$$\frac{k_s}{\omega_b} = \text{const} \quad \implies \quad \text{constant torque tracking error}$$

If k_s increases by $100\times$, ω_b should also increase by $100\times$. Physically limited by motor hardware ($\omega_{b,max} \approx 400$ rad/s for AK60-6).

6 Control Architecture

6.1 Two-Layer Architecture



6.2 Per-Timestep Algorithm

At each simulation step ($\Delta t = 2$ ms), the two blocks execute in sequence:

Block 1 — `ComputedTorqueController` (`controller/controller.py`):

1. **Read plant state** ($q_1, q_2, \dot{q}_1, \dot{q}_2$)
2. **Analytical 2R IK**: $\mathbf{p}_{\text{ref}} \rightarrow (q_1^{\text{des}}, q_2^{\text{des}})$
3. **Differential IK**: $\dot{\mathbf{p}}_{\text{ref}} \xrightarrow{J^{-1}} \dot{\mathbf{q}}_{\text{ref}}, \quad \ddot{\mathbf{p}}_{\text{ref}} \xrightarrow{J^{-1}} \ddot{\mathbf{q}}_{\text{ref}}$
4. **PD + feedforward**: $\mathbf{a}_{\text{des}} = \ddot{\mathbf{q}}_{\text{ref}} + K_p(\mathbf{q}_{\text{des}} - \mathbf{q}) + K_d(\dot{\mathbf{q}}_{\text{ref}} - \dot{\mathbf{q}})$
5. **Inverse dynamics**: $\boldsymbol{\tau}_{\text{des}} = \mathbf{M}(\mathbf{q}) \mathbf{a}_{\text{des}} + \mathbf{h}(\mathbf{q}, \dot{\mathbf{q}})$

Block 2 — `SEACableActuator` (`actuators/sea.py`), delegating to a `MotorDynamics` subclass:

Torque mode (default, `TorqueMotor`):

6. **Motor command**: $\tau_m = \tau_{2,\text{des}}/N$
7. **Cable force**: $\delta = r_p(\theta_m/N - q_2), \quad F = k_s \delta + b_c \dot{\delta}$
8. **Rotor update** (semi-implicit Euler): $\ddot{\theta}_m = (\tau_m - b_m \dot{\theta}_m - r_p F/N)/J_m, \quad \text{update } [\theta_m, \dot{\theta}_m]$
9. **Tension decomposition**: $T_{\text{green}} = \max(F, 0), \quad T_{\text{red}} = \max(-F, 0)$
10. **Apply torque**: $\tau_1 = \tau_{1,\text{des}}$ (rigid), $\tau_2 = r_p \cdot (T_{\text{green}} - T_{\text{red}})$

Position mode (legacy, `PositionServoMotor`):

6. **Motor target** (steady-state inversion): $l_m^{\text{des}} = r_p q_2 + \tau_{2,\text{des}}/(k_s r_p)$
7. **Motor update** (Euler step): $l_m \leftarrow l_m + \Delta t \cdot \omega_b \cdot (l_m^{\text{des}} - l_m)$
8. **Spring force**: $\delta = l_m - r_p q_2, \quad F = k_s \delta + b_c \dot{\delta}$
9. **Tension decomposition**: $T_{\text{green}} = \max(F, 0), \quad T_{\text{red}} = \max(-F, 0)$
10. **Apply torque**: $\tau_1 = \tau_{1,\text{des}}$ (rigid), $\tau_2 = r_p \cdot (T_{\text{green}} - T_{\text{red}})$

Steps 1–5 are the standard CT pipeline (identical to the rigid-tendon controller). Steps 6–10 are the SEA cable layer. Because these are separate Drake `LeafSystems`, any controller that outputs $[\tau_1, \tau_2]$ can be substituted for the CT block.

6.3 Detailed Signal-Flow Block Diagram

Figure ?? shows the complete signal-flow graph for the two-block architecture.

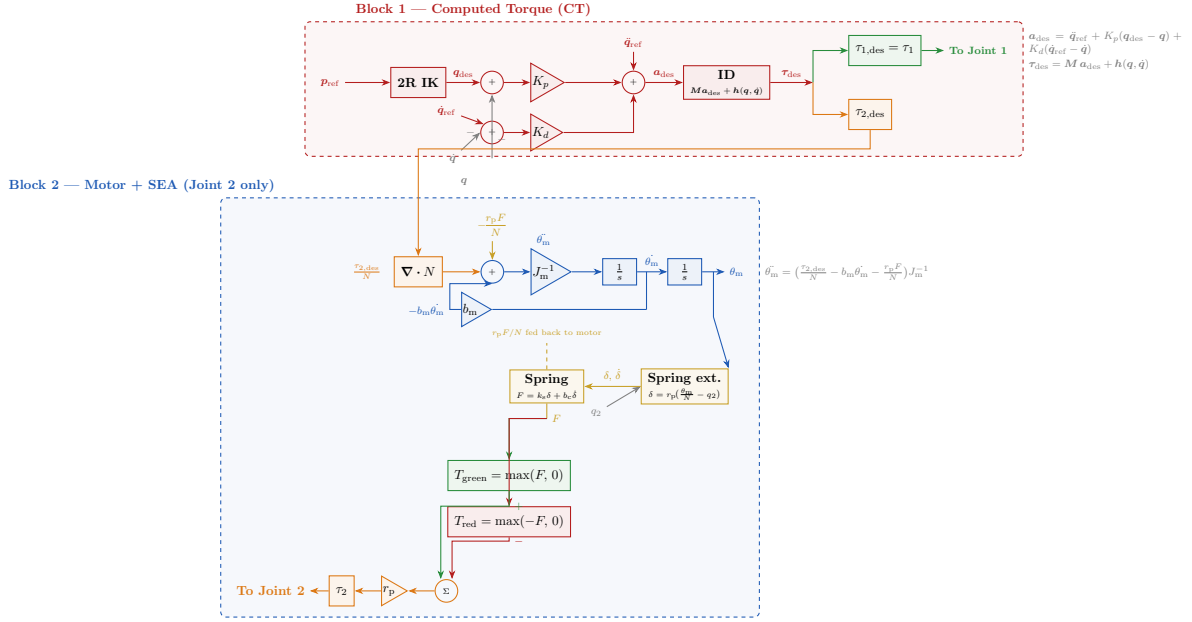


Figure 1: Detailed signal-flow block diagram. **Block 1** (upper, red): Computed-Torque controller—2R inverse kinematics, PD error with feedforward, inverse dynamics. τ_1 goes directly to joint 1 (rigid). $\tau_{2,des}$ enters **Block 2** (lower, blue): motor rotor dynamics with viscous damping b_m , cable spring-damper, unilateral tension split, and pulley radius mapping to joint torque τ_2 .

7 Exosuit Co-Contraction Stiffness

The exosuit adds a **parallel stiffness layer** on joint 2 using two antagonistic cable-spring-motor chains (right and left) routed around a shared centred elbow pulley (Method B, see cable documentation). It does *not* replace the drive SEA; the net torque on joint 2 becomes $\tau_2^{\text{total}} = \tau_2^{\text{drive}} + \tau_{\text{exo}}$.

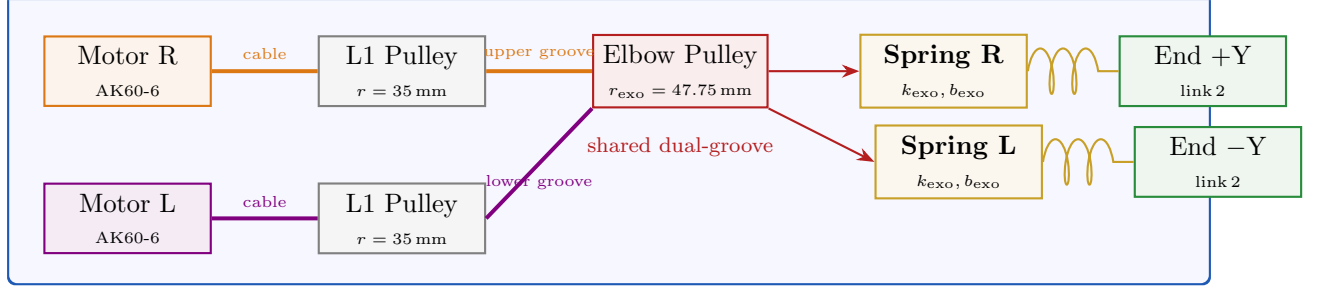
Key idea — symmetric co-contraction. Both exo motors push their springs equally beyond the current joint state, creating a restoring torque proportional to any joint deflection:

$$k_{\text{eff}} = 2 k_{\text{exo}} r_{\text{exo}}^2$$

With $k_{\text{exo}} = 200 \text{ N/m}$ and $r_{\text{exo}} = 47.75 \text{ mm}$ this gives $k_{\text{eff}} \approx 0.91 \text{ Nm/rad}$ of tuneable passive stiffness.

7.1 Physical Cable Routing (Method B — Centred Elbow Pulley)

Exosuit Cable Routing (Joint 2)



The two cables are routed on **separate Z-planes** (3 mm apart) through the same elbow pulley. The **right/orange cable** (upper groove) wraps from the $-Y$ motor start, through a link-1 idler, crosses via an *internal tangent* to the $+Y$ side, and terminates at the link-2 anchor. The **left/purple cable** (lower groove) mirrors this on the opposite side.

Because the elbow pulley is centred on the joint axis, the moment arm is **constant**: $l'_c = r_{\text{exo}}$ for all q_2 .

7.2 Exosuit State Variables

The exosuit adds **four motor-side state variables** (two per antagonistic channel):

Table 5: Exosuit discrete state vector (6D).

Symbol	Description	Units	Index
$\theta_{\text{m,R}}$	Right motor rotor angle	rad	0
$\dot{\theta}_{\text{m,R}}$	Right motor angular velocity	rad/s	1
$\theta_{\text{m,L}}$	Left motor rotor angle	rad	2
$\dot{\theta}_{\text{m,L}}$	Left motor angular velocity	rad/s	3
$q_2^\dagger$	Elbow angle captured at activation (anchor)	rad	4
was_active	Activation flag from previous timestep	bool	5

Motor-side angles. $\theta_{\text{m,R}}$ and $\theta_{\text{m,L}}$ are motor-side (before the gearbox). The joint-side equivalent is θ_{m}/N .

7.3 Spring Extensions and Cable Forces

7.3.1 Right Cable (Upper Groove)

The right cable wraps so that positive motor motion drives $+q_2$:

$$\delta_R = r_{\text{exo}} \left(\frac{\theta_{\text{m,R}}}{N} - q_2 \right) \quad (34)$$

$$\dot{\delta}_R = r_{\text{exo}} \left(\frac{\dot{\theta}_{\text{m,R}}}{N} - \dot{q}_2 \right) \quad (35)$$

7.3.2 Left Cable (Lower Groove)

The left cable wraps in the opposite direction (virtual joint = $-q_2$):

$$\delta_L = r_{\text{exo}} \left(\frac{\theta_{\text{m,L}}}{N} + q_2 \right) \quad (36)$$

$$\dot{\delta}_L = r_{\text{exo}} \left(\frac{\dot{\theta}_{\text{m,L}}}{N} + \dot{q}_2 \right) \quad (37)$$

7.3.3 Unilateral Cable Forces

Cables can only pull ($T \geq 0$):

$$F_R = \max(k_{\text{exo}} \delta_R + b_{\text{exo}} \dot{\delta}_R, 0), \quad F_L = \max(k_{\text{exo}} \delta_L + b_{\text{exo}} \dot{\delta}_L, 0) \quad (38)$$

7.4 Net Exosuit Torque

Using the virtual-work signs ($\partial \delta_R / \partial q_2 = -r_{\text{exo}}$, $\partial \delta_L / \partial q_2 = +r_{\text{exo}}$):

$$\tau_{\text{exo}} = r_{\text{exo}} \cdot (F_R - F_L) \quad (39)$$

Joint 2 receives the **sum** of drive and exo torques:

$$\tau_2^{\text{total}} = \underbrace{r_{\text{p}} \cdot F_{\text{cable}}}_{\text{drive SEA}} + \underbrace{r_{\text{exo}}(F_R - F_L)}_{\text{exosuit}} \quad (40)$$

7.5 Motor Encoder Tracking (Torque Mode)

Each exo motor uses the same 2nd-order MIT torque-mode rotor dynamics as the drive SEA (Section ??), but the **motor command** is generated by an internal **PD encoder-tracking** law rather than by the CT controller.

The exosuit has two operating modes, selectable at runtime:

7.5.1 Mode 1: Transparency (Deactivated)

When deactivated ($\Delta\theta = 0$), both motors continuously track the elbow encoder so that each spring extension stays at zero:

$$\begin{aligned} q_{\text{des,R}} &= q_2 & \dot{q}_{\text{des,R}} &= \dot{q}_2 \\ q_{\text{des,L}} &= -q_2 & \dot{q}_{\text{des,L}} &= -\dot{q}_2 \end{aligned} \quad (41)$$

What happens physically: As the elbow moves (driven by the CT+SEA pipeline), the PD tracker keeps each motor angle at $\theta_{\text{m,R}}/N \approx q_2$ and $\theta_{\text{m,L}}/N \approx -q_2$. The spring extensions therefore remain:

$$\delta_R = r_{\text{exo}} \left(\frac{\theta_{\text{m,R}}}{N} - q_2 \right) \approx 0, \quad \delta_L = r_{\text{exo}} \left(\frac{\theta_{\text{m,L}}}{N} + q_2 \right) \approx 0$$

Because $\delta \approx 0$ in both cables, $F_R \approx 0$ and $F_L \approx 0$, giving $\tau_{\text{exo}} \approx 0$.

Transparency means zero additional stiffness. The exosuit does not resist the elbow motion at all. The motors “follow” the joint, paying out and taking up cable exactly as fast as the elbow moves. The robot behaves as if the exosuit were not present.

In practice, transparency is imperfect: the PD tracker has finite bandwidth, so at high joint velocities there is a small transient $\delta \neq 0$ (and hence a small parasitic torque). With $K_P = 200 \text{ Nm/rad}$ and $\omega_{\text{pd}} \approx 450 \text{ rad/s}$ this residual is negligible for typical trajectory speeds ($\dot{q}_2 < 5 \text{ rad/s}$).

7.5.2 Mode 2: Activated (Co-Contraction)

When activated, a symmetric angular offset $\Delta\theta > 0$ is added to both motor targets. The reference is the CT **desired** joint angle $q_{2,\text{des}}$ (from the trajectory IK), not the actual q_2 (see Section ?? for the rationale):

$$\begin{aligned} q_{\text{des},R} &= q_{2,\text{des}} + \Delta\theta & \dot{q}_{\text{des},R} &= \dot{q}_{2,\text{des}} \\ q_{\text{des},L} &= -q_{2,\text{des}} + \Delta\theta & \dot{q}_{\text{des},L} &= -\dot{q}_{2,\text{des}} \end{aligned} \quad (42)$$

What happens physically: Both motors advance *beyond* the CT desired joint state by $\Delta\theta$. Each motor pays out (or retracts) extra cable $r_{\text{exo}} \Delta\theta$ metres, pretensioning its spring.

When the motors have converged, the steady-state extensions are:

$$\begin{aligned} \delta_R^{\text{ss}} = r_{\text{exo}} \Delta\theta > 0 & \implies F_R = k_{\text{exo}} r_{\text{exo}} \Delta\theta > 0 \\ \delta_L^{\text{ss}} = r_{\text{exo}} \Delta\theta > 0 & \implies F_L = k_{\text{exo}} r_{\text{exo}} \Delta\theta > 0 \end{aligned} \quad (43)$$

Both cables carry **equal pre-tension**. Since $F_R = F_L$, the net torque is:

$$\tau_{\text{exo}}^{\text{ss}} = r_{\text{exo}}(F_R - F_L) = 0$$

At the nominal position the exosuit adds **no net torque**; it only creates latent stiffness that activates when the joint is disturbed.

7.5.3 Motor Reference Port: Tracking $q_{2,\text{des}}$

SEAExoActuator exposes a `q_des` input port (2-element vector) that, when connected, supplies the CT controller's desired joint positions as the motor reference in activated mode.

```

1 # In run_simulation() after building CT and exo systems:
2 builder.Connect(
3     ct.GetOutputPort("joint_positions"), # 2D: [q1_des, q2_des]
4     exo.GetInputPort("q_des"),
5 )

```

Listing 1: Drake diagram wiring: CT desired positions to exosuit reference.

The anchor logic inside `_update_motors()` selects the reference depending on mode:

```

1 if activated:
2     if self._qdes_port.HasValue(context):
3         q_des_vec = self._qdes_port.Eval(context)
4         q2_ref = float(q_des_vec[1]) # track q2_des (CT reference)
5     else:
6         q2_ref = q2_anchor # fallback: frozen anchor
7 else:
8     q2_ref = q2 # transparent: follow actual
9     joint
10 q_des_R = q2_ref + delta_theta # right motor target (joint
11     side)
12 q_des_L = -q2_ref + delta_theta # left motor target (joint side)

```

Listing 2: Reference selection in `SEAExoActuator._update_motors()`.

Table 6: Motor reference selection by mode.

Mode	$q_{2,\text{ref}}$	Spring behaviour	τ_{exo}
Transparent (off)	q_2 (actual)	$\delta \approx 0$ at all times	≈ 0
Activated (on, <code>q_des</code> connected)	$q_{2,\text{des}}$ (CT)	$\delta = r_{\text{exo}}(\pm\Delta\theta \mp (q_2 - q_{2,\text{des}}))$	$-k_{\text{eff}}(q_2 - q_{2,\text{des}})$
Activated (on, fallback)	$q_2^\dagger$ (frozen anchor)	spring pulls back to activation pose	large, fights CT

Why $q_{2,\text{des}}$ instead of q_2 ?

If the reference were the *actual* q_2 , the motor targets would follow the joint, and the spring extensions would stay near $\pm r_{\text{exo}}\Delta\theta$ even during trajectory motion. No net torque would be produced except transiently (relying on motor lag).

With $q_{2,\text{des}}$:

- During **clean tracking** ($q_2 \approx q_{2,\text{des}}$): $\tau_{\text{exo}} \approx 0$ — exosuit is transparent.
- During a **disturbance** ($q_2 \neq q_{2,\text{des}}$): $\tau_{\text{exo}} = -k_{\text{eff}}(q_2 - q_{2,\text{des}})$ acts *immediately*, without relying on motor bandwidth lag.
- The exosuit provides *elastic stiffness* around the CT reference trajectory, not around a fixed pose.

7.5.4 Disturbance Rejection: Restoring the Elbow

Key insight. The motor targets track the CT *desired* joint angle $q_{2,\text{des}}$ (from the trajectory planner and IK solver), **not** the actual joint angle q_2 . When a disturbance deflects q_2 away from $q_{2,\text{des}}$, the motor targets do **not change**—they remain anchored to the CT reference. The joint moves while the motors stay in place, immediately creating asymmetric spring extensions proportional to the deflection:

$$\tau_{\text{exo}} = -k_{\text{eff}}(q_2 - q_{2,\text{des}})$$

This acts as a direct **elastic stiffness** around the CT trajectory reference—no motor bandwidth lag required.

Contrast with earlier implementations.

An earlier version of the code used a *frozen anchor* $q_2^\dagger$ captured at the moment of activation:

$$q_{\text{des,R}} = q_2^\dagger + \Delta\theta \quad (\text{original bug})$$

This caused the exosuit to behave as a stiff spring anchored to the *activation pose*, creating large continuous τ_{exo} that fought the CT controller whenever the arm moved away from the activation configuration (e.g. on circular or rectangular trajectories). The current fix replaces the frozen anchor with $q_{2,\text{des}}$ from the CT's `joint_positions` output port (Section ??).

Pre-condition: nominal steady state at q_2^* . In the activated mode the motor targets are:

$$q_{\text{des,R}} = q_2^* + \Delta\theta, \quad q_{\text{des,L}} = -q_2^* + \Delta\theta \quad (44)$$

Once the motors have converged, their joint-side positions equal the targets:

$$\frac{\theta_{m,R}}{N} = q_2^* + \Delta\theta, \quad \frac{\theta_{m,L}}{N} = -q_2^* + \Delta\theta \quad (45)$$

Substituting into the spring extension formulae (Eqs. ?? and ??) at $q_2 = q_2^*$:

$$\delta_R^{ss} = r_{exo} \left(\frac{\theta_{m,R}}{N} - q_2^* \right) = r_{exo} [(q_2^* + \Delta\theta) - q_2^*] = r_{exo} \Delta\theta \quad (46)$$

$$\delta_L^{ss} = r_{exo} \left(\frac{\theta_{m,L}}{N} + q_2^* \right) = r_{exo} [(-q_2^* + \Delta\theta) + q_2^*] = r_{exo} \Delta\theta \quad (47)$$

Both springs carry the same stretch $r_{exo} \Delta\theta$, so:

$$F_R^{ss} = k_{exo} r_{exo} \Delta\theta = F_L^{ss} \quad (48)$$

The net exo torque at the nominal position is therefore **zero**:

$$\tau_{exo}^{ss} = r_{exo} (F_R^{ss} - F_L^{ss}) = 0 \quad (49)$$

Preloaded but balanced. At the nominal joint position both cables carry equal tension $F_0 = k_{exo} r_{exo} \Delta\theta > 0$ and the net joint torque is zero. The exosuit is preloaded — both springs are stretched — but the antagonistic torques cancel perfectly.

Disturbance event. An external force pushes the elbow from q_2^* to $q_2 = q_2^* + \Delta q$ with $\Delta q > 0$ (further flexion).

Step 1 — Motor targets remain anchored to the CT reference $q_{2,des}$.

Because the motor reference is $q_{2,des}$ (set by the trajectory, not the actual joint), the disturbance *does not change* the motor targets:

$$q_{des,R} = q_2^* + \Delta\theta, \quad q_{des,L} = -q_2^* + \Delta\theta \quad (50)$$

where $q_2^* = q_{2,des}$ is the CT desired angle (unchanged by the disturbance).

The motors, already converged to the nominal positions, remain:

$$\frac{\theta_{m,R}}{N} \approx q_2^* + \Delta\theta, \quad \frac{\theta_{m,L}}{N} \approx -q_2^* + \Delta\theta \quad (51)$$

The restoring torque arises *immediately* (not through motor lag): as soon as q_2 deviates from $q_2^* = q_{2,des}$, the spring extensions become asymmetric.

Step 2 — Spring extensions become asymmetric.

With the joint at $q_2 = q_2^* + \Delta q$ and the motors still at their nominal positions (Eq. ??):

Right cable:

$$\begin{aligned} \delta_R &= r_{exo} \left(\frac{\theta_{m,R}}{N} - q_2 \right) = r_{exo} [(q_2^* + \Delta\theta) - (q_2^* + \Delta q)] \\ &= r_{exo} (\Delta\theta - \Delta q) \quad \underbrace{\leq r_{exo} \Delta\theta}_{\text{right unloads}} \end{aligned} \quad (52)$$

Left cable:

$$\begin{aligned}\delta_L &= r_{\text{exo}} \left(\frac{\theta_{\text{m,L}}}{N} + q_2 \right) = r_{\text{exo}} [(-q_2^* + \Delta\theta) + (q_2^* + \Delta q)] \\ &= r_{\text{exo}} (\Delta\theta + \Delta q) \quad \underbrace{> r_{\text{exo}} \Delta\theta}_{\text{left loads more}}\end{aligned}\tag{53}$$

	Before disturbance	After disturbance ($\Delta q > 0$)
δ_R	$r_{\text{exo}} \Delta\theta$	$r_{\text{exo}} (\Delta\theta - \Delta q) \downarrow$
δ_L	$r_{\text{exo}} \Delta\theta$	$r_{\text{exo}} (\Delta\theta + \Delta q) \uparrow$
$F_R - F_L$	0 (balanced)	$-2 k_{\text{exo}} r_{\text{exo}} \Delta q < 0$

Step 3 — Full derivation of the restoring torque.

Assuming both cables remain taut ($|\Delta q| < \Delta\theta$), the cable forces are:

$$F_R = k_{\text{exo}} \delta_R = k_{\text{exo}} r_{\text{exo}} (\Delta\theta - \Delta q)\tag{54}$$

$$F_L = k_{\text{exo}} \delta_L = k_{\text{exo}} r_{\text{exo}} (\Delta\theta + \Delta q)\tag{55}$$

Net exo torque (expanding fully so all terms are visible):

$$\begin{aligned}\tau_{\text{exo}} &= r_{\text{exo}} (F_R - F_L) \\ &= r_{\text{exo}} [k_{\text{exo}} r_{\text{exo}} (\Delta\theta - \Delta q) - k_{\text{exo}} r_{\text{exo}} (\Delta\theta + \Delta q)] \\ &= k_{\text{exo}} r_{\text{exo}}^2 \underbrace{[(\Delta\theta - \Delta q) - (\Delta\theta + \Delta q)]}_{\Delta\theta \text{ cancels}} \\ &= k_{\text{exo}} r_{\text{exo}}^2 (-2 \Delta q)\end{aligned}\tag{56}$$

$$\boxed{\tau_{\text{exo}} = -2 k_{\text{exo}} r_{\text{exo}}^2 \Delta q = -k_{\text{eff}} \Delta q}\tag{57}$$

Why does $\Delta\theta$ vanish from the restoring torque?

$\Delta\theta$ creates equal preload on *both* cables. When you compute the *difference* $F_R - F_L$, the common preload cancels exactly. The incremental restoring force comes only from the change in imbalance, which is driven by Δq , not by $\Delta\theta$.

Consequence: $\Delta\theta$ is necessary to keep both cables taut (prevents slack), but it does not set the restoring stiffness. Only k_{exo} and r_{exo} determine k_{eff} .

Step 4 — The elbow returns to nominal.

The restoring torque $\tau_{\text{exo}} = -k_{\text{eff}} \Delta q$ acts on the joint *in addition to* whatever the CT+SEA pipeline already provides. It accelerates the elbow back toward q_2^* .

Simultaneously, the motor PD trackers update θ_{m} toward the new (displaced) targets (Eq. ??). As the elbow returns ($\Delta q \rightarrow 0$) two things happen together:

- the spring asymmetry disappears: $\delta_R \rightarrow \delta_L \rightarrow r_{\text{exo}} \Delta\theta$,
- the restoring torque fades: $\tau_{\text{exo}} \rightarrow 0$.

The motors converge to their now-updated targets *after* the joint returns, re-establishing the symmetric preload at the new nominal trajectory point.

Disturbance rejection — full step-by-step.

1. **Nominal:** both cables carry equal preload $F_0 = k_{\text{exo}} r_{\text{exo}} \Delta\theta$. Net torque = 0.
2. **Disturbance** displaces q_2 by $\Delta q > 0$.
3. **Motors lag:** joint moves, motors are still at old positions.
4. **Asymmetric stretch:** right cable unloads ($\delta_R \downarrow$), left cable loads ($\delta_L \uparrow$).
5. **Restoring torque:** $\tau_{\text{exo}} = -k_{\text{eff}} \Delta q$ (opposes displacement).
6. **Return:** exo torque drives elbow back; as $\Delta q \rightarrow 0$, $\tau_{\text{exo}} \rightarrow 0$ and symmetric preload is restored.

Analogy: two preloaded antagonistic muscles around the elbow joint. If the arm is pushed, one muscle tightens and the other relaxes; the net difference restores the arm.

Cable slack limit. The linear law (Eq. ??) holds only while **both cables remain taut**, i.e. $\delta_R \geq 0 \Rightarrow \Delta q \leq \Delta\theta$. If the disturbance is large enough to slack one cable, the piecewise torque law applies:

$$\tau_{\text{exo}} = \begin{cases} -k_{\text{eff}} \Delta q & |\Delta q| \leq \Delta\theta \quad (\text{bilateral stiffness}) \\ +k_{\text{exo}} r_{\text{exo}}^2 (\Delta\theta - \Delta q) & \Delta q > +\Delta\theta \quad (\text{right slack, left only}) \\ -k_{\text{exo}} r_{\text{exo}}^2 (\Delta\theta + \Delta q) & \Delta q < -\Delta\theta \quad (\text{left slack, right only}) \end{cases}$$

Beyond the slack threshold the effective stiffness halves and becomes **unilateral**. Increasing $\Delta\theta$ widens the bilateral zone without changing its slope (since k_{eff} is independent of $\Delta\theta$).

7.5.5 PD Tracking Torque (Joint Side)

The motor torque command fed to each rotor is a saturated PD:

$$\tau_{m,i} = \text{clip}\left(K_P \left(q_{\text{des},i} - \frac{\theta_{m,i}}{N}\right) + K_D \left(\dot{q}_{\text{des},i} - \frac{\dot{\theta}_{m,i}}{N}\right), -\tau_{\text{max}}, \tau_{\text{max}}\right) \quad (58)$$

where $K_P = 200 \text{ Nm/rad}$, $K_D = 2 \text{ Nm} \cdot \text{s/rad}$, $\tau_{\text{max}} = 9 \text{ Nm}$ (AK60-6 peak).

7.5.6 Rotor Dynamics (Per Motor, Sub-Stepped)

Each rotor integrates the same 2nd-order ODE as the drive motor (Eq. ??):

$$J_m \ddot{\theta}_{m,i} = \frac{\tau_{m,i}}{N} - b_m \dot{\theta}_{m,i} - \frac{r_{\text{exo}} \cdot F_i}{N} \quad (59)$$

Sub-stepping is essential. The PD natural frequency $\omega_{\text{pd}} = \sqrt{K_P/(J_m N^2)} \approx 450 \text{ rad/s}$ is much higher than the drive motor's spring resonance. Without sub-stepping

($N_{\text{sub}} = 10$ per Δt), the integrator becomes unstable.

7.6 Effective Co-Contraction Stiffness

When both exo motors track $q_2 \pm \Delta\theta$ perfectly, a small joint perturbation Δq from the current position produces:

$$\delta_R = r_{\text{exo}}(\Delta\theta - \Delta q) \quad (60)$$

$$\delta_L = r_{\text{exo}}(\Delta\theta + \Delta q) \quad (61)$$

so the restoring torque is:

$$\begin{aligned} \tau_{\text{exo}} &= r_{\text{exo}}(k_{\text{exo}} r_{\text{exo}}(\Delta\theta - \Delta q) - k_{\text{exo}} r_{\text{exo}}(\Delta\theta + \Delta q)) \\ &= -2 k_{\text{exo}} r_{\text{exo}}^2 \Delta q \end{aligned} \quad (62)$$

Effective co-contraction stiffness:

$$k_{\text{eff}} = 2 k_{\text{exo}} r_{\text{exo}}^2 \quad (63)$$

This is **independent of $\Delta\theta$** (the offset sets pre-tension, not stiffness) and **independent of q_2** (constant moment arm, Method B).

Table 7: Effective stiffness for typical exo spring values ($r_{\text{exo}} = 47.75$ mm).

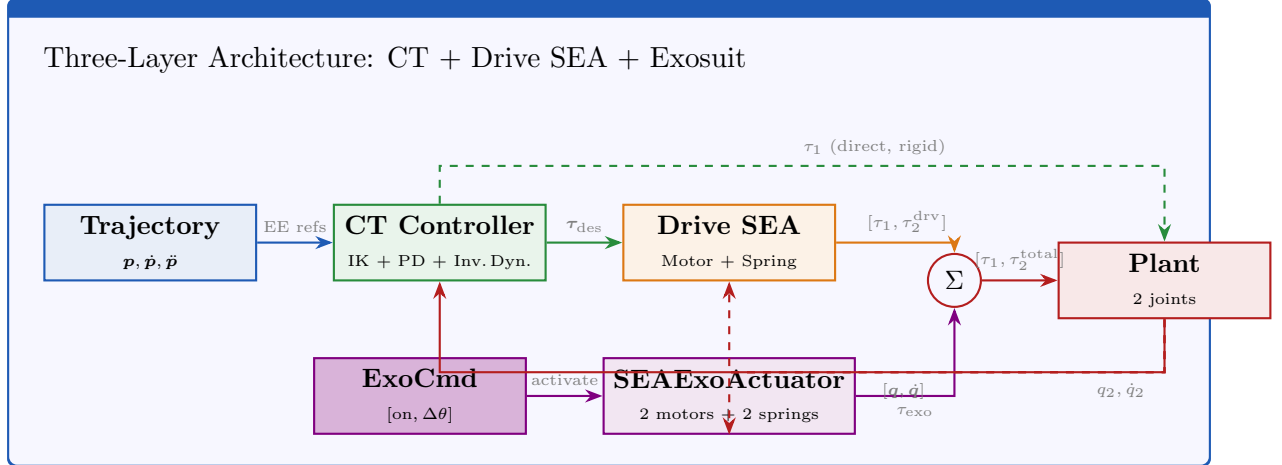
k_{exo} [N/m]	k_{eff} [Nm/rad]
60	0.27
100	0.46
200	0.91
500	2.28

7.7 Exosuit Parameters

Table 8: Default exosuit parameters (SEAExoActuator constructor).

Parameter	Symbol	Default	Unit
<i>Cable & Spring</i>			
Spring stiffness	k_{exo}	200	N/m
Cable damping	b_{exo}	2.0	N·s/m
Elbow pulley radius	r_{exo}	47.75	mm
<i>Motor (CubeMars AK60-6, each)</i>			
Gear ratio	N	6	—
Rotor inertia	J_{m}	3.32×10^{-5}	kg·m ²
Viscous damping	b_{m}	derived	N·m·s/rad
Peak torque	τ_{max}	9.0	Nm
<i>PD Tracking</i>			
Position gain	K_P	200	Nm/rad
Velocity gain	K_D	2.0	Nm·s/rad
Sub-steps per Δt	N_{sub}	10	—
<i>Activation</i>			
Co-contraction offset	$\Delta\theta$	0.1	rad
Activation time	t_{act}	5.0	s

7.8 Exosuit Control Architecture



Additional wire (not shown): the CT controller also sends $q_{2,\text{des}}$ to the exosuit via its q_{des} input port (Section ??). This allows the exosuit to track the desired trajectory rather than a frozen anchor.

7.9 Detailed Signal-Flow Block Diagram (Exosuit Only)

Figure ?? shows the signal-flow graph for a **single exo motor–spring channel** (right cable shown; the left is identical with virtual joint $-q_2$).

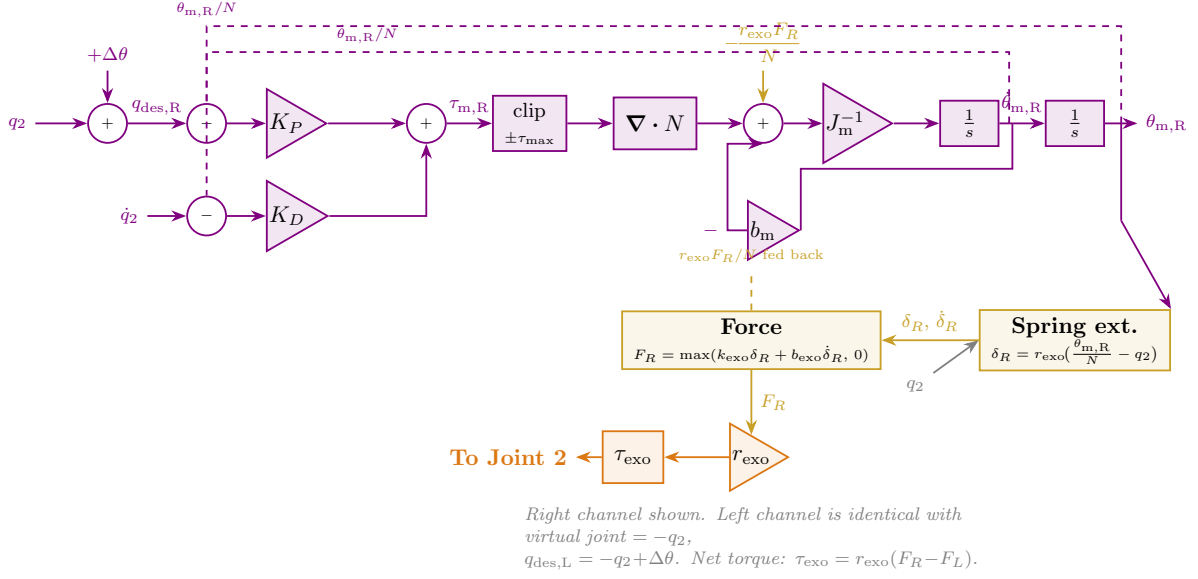


Figure 2: Signal-flow block diagram for one exosuit motor-spring channel (right cable). A PD encoder-tracking law generates motor torque $\tau_{m,R}$; the 2nd-order rotor integrates with viscous damping and spring-load feedback; the unilateral spring force F_R maps through the elbow pulley radius to a torque contribution. The left channel mirrors this with virtual joint $-q_2$. The net exosuit torque is $\tau_{exo} = r_{exo}(F_R - F_L)$.

7.10 Per-Timestep Algorithm (Torque Mode)

At each simulation step ($\Delta t = 2 \text{ ms}$), the exosuit executes in parallel with the drive SEA (both read the same plant state):

Block 3 — SEAExoActuator (actuators/sea_exo.py):

1. **Read activation command:** [on/off, $\Delta\theta$]
2. **Read joint state:** extract $(q_2, \dot{q}_2)$ from plant
3. **Compute motor targets** (use $q_{2,des}$ from q_des port when activated; use q_2 when transparent):

$$q_{des,R} = q_{2,ref} + \Delta\theta, \quad q_{des,L} = -q_{2,ref} + \Delta\theta$$

where $q_{2,ref} = q_{2,des}$ if activated, q_2 if transparent.

4. **Sub-stepped loop** ($N_{sub} = 10$ iterations):
 - (a) PD tracking torque (Eq. ??): $\tau_{m,i}$ for each motor
 - (b) Rotor dynamics (Eq. ??): update $[\theta_{m,i}, \dot{\theta}_{m,i}]$
5. **Spring forces** (Eq. ??): F_R, F_L (unilateral)
6. **Net exo torque** (Eq. ??): $\tau_{exo} = r_{exo}(F_R - F_L)$
7. **Clamp and output:** $\tau_{exo} \leftarrow \text{clip}(\tau_{exo}, -\tau_{max}, \tau_{max})$

The output τ_{exo} is summed with the drive torques in an **ActuationSum** adder block before reaching the plant.

7.11 Diagnostics Output

The diagnostics output port exports a 10-element vector for logging and plotting:

Table 9: Exosuit diagnostics vector (10D).

Index	Quantity	Unit
0	δ_R — right spring extension	m
1	δ_L — left spring extension	m
2	F_R — right cable force	N
3	F_L — left cable force	N
4	$\theta_{m,R}/N$ — right motor joint-side pos	rad
5	$\theta_{m,L}/N$ — left motor joint-side pos	rad
6	$\dot{\theta}_{m,R}/N$ — right motor joint-side vel	rad/s
7	$\dot{\theta}_{m,L}/N$ — left motor joint-side vel	rad/s
8	τ_{exo} — net exo torque	Nm
9	activated (1.0 or 0.0)	—

7.12 Summary of Exosuit Equations

Exosuit Equation Summary (Torque Mode) **Motor target** (activated, reference = CT desired $q_{2,\text{des}}$):

$$q_{\text{des},R} = q_{2,\text{des}} + \Delta\theta \quad q_{\text{des},L} = -q_{2,\text{des}} + \Delta\theta$$

PD tracking:

$$\tau_{m,i} = \text{clip}\left(K_P(q_{\text{des},i} - \theta_{m,i}/N) + K_D(\dot{q}_{\text{des},i} - \dot{\theta}_{m,i}/N), \pm\tau_{\text{max}}\right)$$

Rotor dynamics:

$$J_m \ddot{\theta}_{m,i} = \frac{\tau_{m,i}}{N} - b_m \dot{\theta}_{m,i} - \frac{r_{\text{exo}} F_i}{N}$$

Spring extensions:

$$\delta_R = r_{\text{exo}} \left(\frac{\theta_{m,R}}{N} - q_2 \right), \quad \delta_L = r_{\text{exo}} \left(\frac{\theta_{m,L}}{N} + q_2 \right)$$

Cable forces (unilateral):

$$F_i = \max(k_{\text{exo}} \delta_i + b_{\text{exo}} \dot{\delta}_i, 0)$$

Net exo torque:

$$\tau_{\text{exo}} = r_{\text{exo}} (F_R - F_L)$$

Effective stiffness (co-contraction):

$$k_{\text{eff}} = 2 k_{\text{exo}} r_{\text{exo}}^2$$

8 Drake Implementation

8.1 Source File

`script_cup_manipulator_pendulam_tendon_with_spring_sea_pydrake.py` (~900 lines) uses a **two-block architecture**: control logic in `ComputedTorqueController` and actuator physics in `SEACableActuator`, wired together via a Drake diagram.

Table 10: Key classes and functions.

Name	Role
<code>ComputedTorqueController</code>	Drake <code>LeafSystem</code> (controller/controller.py). IK + PD + inverse dynamics. Outputs τ_{des} .
<code>SEACableActuator</code>	Drake <code>LeafSystem</code> (actuators/sea.py). Delegates to <code>MotorDynamics</code> + spring-damper cable model.
<code>MotorDynamics</code>	ABC (actuators/motor_dynamics.py). Subclasses: <code>TorqueMotor</code> (2nd-order), <code>PositionServoMotor</code> (1st-order).
<code>_build_rect_trajectory</code>	C^2 rectangular EE trajectory with speed-profiled corners.
<code>_build_move_to_start</code>	Cubic-Hermite approach preamble.
<code>run_simulation()</code>	Build, wire, run, collect logs for one SEA config.
<code>plot_sea_results()</code>	3×3 diagnostic panel + EE path overlay.
<code>main()</code>	CLI dispatch, optional <code>--compare</code> mode.

8.2 SEACableActuator — Pure Actuator LeafSystem

The `SEACableActuator` (actuators/sea.py) is a pure physics model with no control law. It accepts desired torques $[\tau_1, \tau_2]$ and outputs actual torques after motor + spring dynamics. Motor dynamics are delegated to a pluggable `MotorDynamics` object (actuators/motor_dynamics.py).

```

1 class MotorMode(Enum):
2     TORQUE = "torque"          # 2nd-order rotor dynamics (default)
3     POSITION = "position"       # 1st-order position servo (legacy)
4
5 class MotorDynamics(ABC):      # shared interface
6     def step(self, state, tau2_des, q2, q2_dot) -> np.ndarray: ...
7     def compute_spring_force(self, state, ...) -> tuple: ...
8
9 class TorqueMotor(MotorDynamics): #  $J_m * \theta_{ddot} = \tau_m - b_m$ 
10    *  $\theta_{dot} - r_p * F / N$ 
11
12 class PositionServoMotor(MotorDynamics): #  $\dot{l} = \omega_b * (l_{des} - l)$ 

```

Listing 3: Motor dynamics class hierarchy (actuators/motor_dynamics.py).

```

1 class TorqueMotor(MotorDynamics):
2     def _cable_force_raw(self, theta_m, theta_m_dot, q2, q2_dot):
3         delta_lin = self._r_p * (theta_m / self._N - q2)
4         delta_dot_lin = self._r_p * (theta_m_dot / self._N - q2_dot)
5         F_raw = self._k_s * delta_lin + self._b_c * delta_dot_lin
6         return F_raw, delta_lin
7

```

```

8     def step(self, state, tau2_des, q2, q2_dot):
9         theta_m, theta_m_dot = state[0], state[1]
10        tau_m = tau2_des / self._N
11        F_raw, _ = self._cable_force_raw(theta_m, theta_m_dot, q2,
12                                         q2_dot)
13        tau_spring_motor = self._r_p * F_raw / self._N
14        theta_m_ddot = (tau_m - self._b_m * theta_m_dot
15                        - tau_spring_motor) / self._J_m
16        # Semi-implicit Euler (velocity first, then position)
17        theta_m_dot_new = theta_m_dot + self._dt * theta_m_ddot
18        theta_m_new = theta_m + self._dt * theta_m_dot_new
19        return np.array([theta_m_new, theta_m_dot_new])

```

Listing 4: TorqueMotor: cable force in linear coordinates + semi-implicit Euler.

► Equations implemented by the above code

$$\begin{aligned}
 \delta &= r_p \cdot (\theta_m / N - q_2) && \text{(spring extension [m])} \\
 \dot{\delta} &= r_p \cdot (\dot{\theta}_m / N - \dot{q}_2) && \text{(extension rate)} \\
 F_{\text{raw}} &= k_s \delta + b_c \dot{\delta} && \text{(spring-damper)} \\
 F_{\text{cable}} &= T_{\text{green}} - T_{\text{red}} = F_{\text{raw}} && \text{(net antagonistic force)}
 \end{aligned}$$

```

1  def _update_motor(self, context, discrete_state):
2      motor_state = ... # read from Drake discrete state
3      tau_des = self._tau_port.Eval(context)
4      q, q_dot = self._read_joint_state(context)
5      new_state = self._motor.step(motor_state, tau_des[1], q[1], q_dot
6                                   [1])
7      # write new_state back to Drake discrete state
8
9  def _calc_actuation(self, context, output):
10     tau_des = self._tau_port.Eval(context)
11     motor_state = ... # read from Drake discrete state
12     q, q_dot = self._read_joint_state(context)
13     F_cable, _, _, _, _ = self._motor.compute_spring_force(
14         motor_state, tau_des[1], q[1], q_dot[1])
15     tau_out = np.array([
16         tau_des[0], # J1: pass-through (rigid)
17         self._r_p * F_cable, # J2: SEA cable spring
18     ])
19     output.SetFromVector(np.clip(tau_out, -self._tau_max, self._tau_max))

```

Listing 5: SEACableActuator delegates to MotorDynamics (actuators/sea.py).

► Equations implemented by the above code

$$\begin{array}{ll}
\text{Torque mode: } J_m \ddot{\theta}_m = \tau_m - b_m \dot{\theta}_m - r_p F / N & (\text{rotor EOM}) \\
\text{Position mode: } l_m[k+1] = l_m[k] + \Delta t \cdot \omega_b \cdot (l_m^{\text{des}} - l_m[k]) & (\text{motor servo}) \\
\tau_1 = \tau_{1,\text{des}} & (\text{rigid J1}) \\
\tau_2 = r_p \cdot F_{\text{cable}} & (\text{SEA J2})
\end{array}$$

8.3 Diagnostics Output Port

The `SEACableActuator` exposes an 8-element diagnostics vector for logging:

Table 11: Diagnostics vector layout (8 elements). Slots 0–1 depend on motor mode.

Idx	Name	Symbol	Torque mode	Position mode
0	Motor state (primary)	—	θ_m / N [rad]	l_m [m]
1	Motor state (secondary)	—	$\dot{\theta}_m / N$ [rad/s]	l_m^{des} [m]
2	Spring extension	δ		m
3	Net cable force	F_{cable}		N
4	CT desired τ_1	$\tau_{1,\text{des}}$		Nm
5	CT desired τ_2	$\tau_{2,\text{des}}$		Nm
6	Green cable tension	T_{green}		N
7	Red cable tension	T_{red}		N

9 Spring Visualization

The Meshcat 3D visualization draws the cable routing with pulleys, straight cable segments, and a helical spring coil on the last segment of each cable route.

9.1 Dynamic Spring Length

The spring visualization dynamically reflects the actual physics state. The helper function `draw_cables()` in `project_utils/viz_cables.py` accepts a `spring_extension` parameter:

```

1 def draw_cables(meshcat, plant, plant_context, manipulator, rig,
2                 cable_radius=0.0005, n_arc_pts=32,
3                 spring_extension=0.0):
4     # ...
5     # In the last cable segment:
6     rest_len = spring.rest_length
7     actual_len = rest_len + abs(spring_extension)
8     sf = np.clip(actual_len / seg_len, 0.05, 0.90)

```

Listing 6: Dynamic spring fraction from physics state.

► Equations implemented by the above code

The visual spring fraction of the cable segment is:

$$f_{\text{spring}} = \text{clamp}\left(\frac{L_{\text{rest}} + |\delta|}{L_{\text{segment}}}, 0.05, 0.90\right)$$

When $\delta = 0$ (no tension), the spring is at its rest fraction. Under load, $|\delta| > 0$ and the

coil visually stretches, with the remaining segment drawn as straight cable on each side.

9.2 Real-Time Update

During each simulation visualization tick, the spring extension δ is read from the SEA actuator's diagnostics port and passed to the drawing function:

```

1 def _viz_tick():
2     _ctx = simulator.get_mutable_context()
3     _pc = plant.GetMyMutableContextFromRoot(_ctx)
4     _sea_ctx = sea.GetMyMutableContextFromRoot(_ctx)
5     _diag = sea.GetOutputPort("diagnostics").Eval(_sea_ctx)
6     _delta = _diag[2] # spring extension delta [m]
7     manipulator.compute_tangents(plant, _pc)
8     draw_cables(meshcat, plant, _pc, manipulator, _rig,
9                 spring_extension=_delta)

```

Listing 7: Visualization tick with live spring extension.

10 Comparison: Rigid vs. Compliant Tendon

10.1 Rigid Tendon (Standard CT)

In the rigid-tendon model (companion document):

- $\tau_2 = \tau_{2,\text{des}}$ — torque applied instantaneously
- No spring state, no motor dynamics
- Cable tensions are post-hoc logging: $T = \tau_2/r_p$

10.2 SEA Tendon (This Document)

- $\tau_2 = r_p \cdot F_{\text{cable}}$ — torque mediated by spring + motor
- Motor states: torque mode $[\theta_m, \dot{\theta}_m]$ (2 states), position mode $[l_m]$ (1 state)
- Cable tensions are *physically meaningful*: they determine the actual force
- Torque lag increases with softer spring (lower k_s) or slower motor

Table 12: Rigid vs. SEA comparison.

Property	Rigid Tendon	SEA Cable
Joint-2 torque source	CT output directly	Spring force $\times r_p$
Cable dynamics	None	Torque: $J_m \ddot{\theta}_m = \tau_m - b_m \dot{\theta}_m - r_p F/N$ Position: $\dot{l}_m = \omega_b (l_m^{\text{des}} - l_m)$
Spring state	N/A	$\delta = l_m - r_p q_2$
Antagonistic cables	Post-hoc only	Physics: bidirectional
Torque bandwidth	Infinite	Limited by k_s , motor dynamics
Tracking accuracy	Best possible	Degrades with soft spring
Extra parameters	0	Torque: 3 (k_s, b_c, J_m); Position: 3 (k_s, b_c, ω_b)

11 Command-Line Interface

```

1 # Default spring (k_s=20 N/m), default CT gains
2 python script_cup_manipulator_pendulam_tendon_with_spring_sea_pydrake
  .py
3
4 # Very soft spring (high lag)
5 python script_cup_manipulator_pendulam_tendon_with_spring_sea_pydrake
  .py --spring-stiffness 30
6
7 # Stiff spring (near-rigid, matches CT-only behaviour)
8 python script_cup_manipulator_pendulam_tendon_with_spring_sea_pydrake
  .py --spring-stiffness 5000
9
10 # Overlay compliant vs rigid on the same plot
11 python script_cup_manipulator_pendulam_tendon_with_spring_sea_pydrake
  .py --compare
12
13 # Custom CT gains + fast motor
14 python script_cup_manipulator_pendulam_tendon_with_spring_sea_pydrake
  .py \
15     --spring-stiffness 200 --motor-bandwidth 60 \
16     --ct-kp 400 --ct-kd 80

```

Listing 8: Example usage.

Table 13: SEA-specific CLI arguments.

Flag	Symbol	Default	Description
--sea-mode	—	torque	Motor dynamics mode (torque or position)
--spring-stiffness	k_s	3000 N/m	Cable spring stiffness
--cable-damping	b_c	2.0 N·s/m	Cable dashpot damping
--motor-bandwidth	ω_b	100 rad/s	Position-mode servo bandwidth (conservative; open-loop $1/\tau$)
--compare	—	off	Run compliant + rigid, overlay plots
--compare-rigid-ks	—	5000 N/m	Stiffness for “rigid” comparison

12 Computed Torque Controller — Inner Layer

The SEA cable model (Section ??) sits *between* the controller and the plant. The **Computed Torque (CT)** controller is the inner layer that decides what torque *should* be applied. This section documents the complete CT pipeline as implemented in the `_solve()` method of `ComputedTorqueController` (`controller/controller.py`).

12.1 Inverse Kinematics: Task Space $\rightarrow$ Joint Space

Given an end-effector reference position $\mathbf{p}_{\text{ref}} = [x, y]^T$, the controller solves the analytical 2R inverse kinematics:

$$q_2 = \pm \arccos\left(\frac{x^2 + y^2 - L_1^2 - L_2^2}{2 L_1 L_2}\right), \quad q_1 = \arctan 2(y, x) - \arctan 2(L_2 \sin q_2, L_1 + L_2 \cos q_2) \quad (64)$$

Key implementation details:

- **Warm-starting:** The previous IK solution seeds the next iteration to maintain branch consistency (no elbow flips between timesteps).
- **Workspace clamping:** Target points outside the reachable annulus $r \in [|L_1 - L_2|, L_1 + L_2]$ are projected onto the nearest reachable boundary.
- **Fallback:** If IK fails, the last-known-good solution is held.

```

1 seed = self._last_q_des if np.any(self._last_q_des != 0) else q
2 q_des, ok = self._manip.ik._solve_2r_core(self._L1, self._L2, ee_des,
      seed)
3 if ok:
4     self._last_q_des = q_des.copy()
5 else:
6     q_des = self._last_q_des.copy()    # hold last known good

```

Listing 9: IK warm-start in ComputedTorqueController.solve().

12.2 Differential IK: Velocity & Acceleration References

The end-effector velocity and acceleration references are mapped to joint space via the analytical Jacobian:

$$\mathbf{J}(\mathbf{q}_{\text{des}}) = \begin{bmatrix} -L_1 s_1 - L_2 s_{12} & -L_2 s_{12} \\ L_1 c_1 + L_2 c_{12} & L_2 c_{12} \end{bmatrix} \quad (65)$$

where $s_1 = \sin q_1$, $c_1 = \cos q_1$, $s_{12} = \sin(q_1 + q_2)$, $c_{12} = \cos(q_1 + q_2)$. The joint-space references are:

$$\dot{\mathbf{q}}_{\text{ref}} = \mathbf{J}^{-1} \dot{\mathbf{p}}_{\text{ref}}, \quad \ddot{\mathbf{q}}_{\text{ref}} = \mathbf{J}^{-1} \ddot{\mathbf{p}}_{\text{ref}} \quad (66)$$

$\mathbf{J}^{-1}$ is computed via pseudo-inverse (`np.linalg.pinv`). Near kinematic singularities ($\det \mathbf{J} \approx 0$), the inverse degrades gracefully; however, very large joint velocities can still occur. The torque clamp (τ_{max}) prevents damage.

12.3 PD + Feedforward Desired Acceleration

The controller computes a desired joint acceleration that combines **feedforward** (reference acceleration) with **feedback** (PD error correction):

$$\mathbf{a}_{\text{des}} = \ddot{\mathbf{q}}_{\text{ref}} + K_p (\mathbf{q}_{\text{des}} - \mathbf{q}) + K_d (\dot{\mathbf{q}}_{\text{ref}} - \dot{\mathbf{q}}) \quad (67)$$

Because the system is feedback-linearized (see next section), the closed-loop error dynamics become a decoupled linear 2nd-order system:

$$\ddot{\mathbf{e}} + K_d \dot{\mathbf{e}} + K_p \mathbf{e} = \mathbf{0} \quad \implies \quad \text{poles at } -\zeta \omega_n \pm \omega_n \sqrt{\zeta^2 - 1} \quad (68)$$

Table 14: CT gain interpretation (second-order error dynamics).

Gain	Relation	Meaning
K_p [s ⁻²]	$\omega_n = \sqrt{K_p}$	Natural frequency: $K_p = 100 \rightarrow \omega_n = 10$ rad/s
K_d [s ⁻¹]	$\zeta = K_d / (2\sqrt{K_p})$	Damping ratio: $K_d = 40, K_p = 100 \rightarrow \zeta = 2.0$ (overdamped)

12.4 Inverse Dynamics: $\tau = Ma + h$

Drake's `CalcInverseDynamics` computes the joint torques needed to achieve $\mathbf{a}_{\text{des}}$:

$$\tau_{\text{des}} = M(q) \mathbf{a}_{\text{des}} + \mathbf{h}(q, \dot{q}) \quad (69)$$

where $M(q)$ is the 2×2 mass matrix and $\mathbf{h}(q, \dot{q})$ is the bias vector (Coriolis + gravity).

```

1 # Map user-order [q1, q2] acceleration to Drake velocity-vector order
2 vdot_des = np.zeros(self._nv)
3 vdot_des[self._v_idx[0]] = a_des_user[0]
4 vdot_des[self._v_idx[1]] = a_des_user[1]
5
6 # tau = M(q) * a_des + h(q, qdot)
7 self._forces.SetZero()
8 tau_full = self._plant.CalcInverseDynamics(
9     self._plant_ctx, vdot_des, self._forces,
10 )
11 tau_des = np.array([tau_full[self._v_idx[0]], tau_full[self._v_idx
    [1]]])

```

Listing 10: Computed torque: inverse dynamics via Drake.

► Equations implemented by the above code

Drake's `CalcInverseDynamics(ctx, vdot, external_forces)` computes:

$$\tau = M(q) \ddot{q} + C(q, \dot{q}) \dot{q} + g(q)$$

With `external_forces` set to zero, the full nonlinear dynamics (mass, Coriolis, gravity) are cancelled, and the plant behaves as a double integrator under the desired acceleration.

12.5 Motor Target: Steady-State Inversion

After computing $\tau_{2,\text{des}}$, the controller calculates the motor cable position needed to produce that torque at steady state:

$$l_{\text{m}}^{\text{des}} = r_{\text{p}} q_2 + \frac{\tau_{2,\text{des}}}{k_{\text{s}} r_{\text{p}}} \quad (70)$$

```

1 l_m_des = self._r_p * q[1] + tau_des[1] / (self._k_s * self._r_p)

```

Listing 11: Motor target from desired torque (steady-state spring inversion).

► Equations implemented by the above code

At steady state ($\dot{\delta} = 0$, $l_{\text{m}} = l_{\text{m}}^{\text{des}}$):

$$\begin{aligned} \delta_{\text{ss}} &= \frac{\tau_{2,\text{des}}}{k_{\text{s}} r_{\text{p}}} \\ l_{\text{m}}^{\text{des}} &= r_{\text{p}} q_2 + \delta_{\text{ss}} \end{aligned}$$

The motor must wind the cable past the joint-angle position by exactly the spring deflection needed.

12.6 Per-Timestep Cache

The `ComputedTorqueController` caches its full solve result per simulation timestep. Since Drake evaluates output ports lazily (actuation, joint_positions, torques_raw, and cable_tensions may each trigger `_solve()`), caching avoids $4\times$ redundant IK + inverse-dynamics calls:

```

1 def _solve(self, context):
2     t = context.get_time()
3     if t == self._t_cache and self._cache is not None:
4         return self._cache
5     # ... full IK + CT + motor-target computation ...
6     self._t_cache = t
7     self._cache = (tau_des, l_m_des, q, q_dot, q_des)
8     return self._cache

```

Listing 12: Timestep-keyed cache avoids redundant solves.

13 Trajectory Generation

The simulation uses C^2 -continuous EE trajectories in task space.

13.1 Rectangular Trajectory with Speed Profiling

`_build_rect_trajectory()` creates a looping rectangular path with **velocity profiling**: the EE slows down at corners and speeds up on straight segments.

1. **Arc-length parameterization**: N uniformly-spaced points along the rectangle perimeter $P = 2(W + H)$.
2. **Corner detection**: For each sample, compute the minimum arc-length distance to any corner.
3. **Speed blend**: Hermite interpolation between v_{corner} and v_{max} :

$$v(s) = v_{\text{corner}} + (v_{\text{max}} - v_{\text{corner}})t^2(3 - 2t), \quad t = \text{clamp}\left(\frac{d_{\text{corner}}(s)}{d_{\text{blend}}}, 0, 1\right)$$

4. **Non-uniform timestamps**: Integrate $dt = ds/v(s)$ via trapezoidal rule, then rescale to match the desired lap duration.
5. C^2 **cubic spline**: Drake’s `CubicWithContinuousSecondDerivatives` produces position, velocity, and acceleration that are all continuous.
6. **Workspace clamping**: Points outside the reachable annulus are projected inward to $0.97 r_{\text{max}}$.

Table 15: Rectangle trajectory CLI parameters.

Flag	Default	Units	Description
<code>--traj-x-range</code>	0.30, 0.55	m	X bounds $[x_{\min}, x_{\max}]$
<code>--traj-y-range</code>	-0.12, 0.12	m	Y bounds $[y_{\min}, y_{\max}]$
<code>--traj-N</code>	80	—	Waypoints per lap
<code>--traj-v-max</code>	0.10	m/s	Straight-segment speed
<code>--traj-v-corner</code>	0.02	m/s	Corner speed
<code>--traj-corner-blend</code>	0.3	—	Fraction of $\min(W, H)$ for blend zone

13.2 Move-to-Start Preamble

Before the looping trajectory begins, a **cubic-Hermite approach** moves the EE from its initial position to the first waypoint:

$$\mathbf{p}(t) = \text{CubicHermite}([0, T_{\text{move}}], [\mathbf{p}_{\text{start}}, \mathbf{p}_{\text{end}}], [\mathbf{0}, \dot{\mathbf{p}}_{\text{end}}]) \quad (71)$$

The EE starts from rest ($\dot{\mathbf{p}} = \mathbf{0}$) and arrives at the first waypoint with the velocity that the looping trajectory expects, ensuring a smooth handoff. A custom `PreambleSrc` LeafSystem switches from the preamble to the looping trajectory at $t = T_{\text{move}}$.

13.3 Additional Trajectory Types

The `--traj-type` CLI argument selects from four EE trajectory shapes. All types use the same move-to-start preamble and workspace clamping helper (`_clamp_to_reach`) to keep waypoints inside the reachable annulus.

Table 16: Trajectory type CLI options (`--traj-type`).

Value	Shape	Parametric equations
<code>rect</code> (default)	Rectangle	See Section ??
<code>circle</code>	Circle	$x = c_x + R \cos s, y = c_y + R \sin s$
<code>figure8</code>	Figure-of-eight	$x = c_x + A_x \sin s, y = c_y + A_y \sin s \cos s$
<code>line</code>	Back-and-forth	$x = c_x, y = y_{\min} + \frac{\Delta y}{2}(1 - \cos \Omega t)$

13.3.1 Circle Trajectory

A uniform-speed circle centred at (c_x, c_y) with radius R :

$$x(s) = c_x + R \cos s, \quad y(s) = c_y + R \sin s, \quad s \in [0, 2\pi) \quad (72)$$

The radius is set by `--traj-radius` R ; it defaults to half the minimum range extent if not specified. Speed is uniform ($\|\dot{\mathbf{p}}\| = v_{\max}$) throughout—no corner blending is needed. The centroid (c_x, c_y) is the midpoint of the configured x - and y -ranges.

13.3.2 Figure-of-Eight Trajectory (Lemniscate-of-Gerono)

A figure-of-eight (Lemniscate-of-Gerono) centred at (c_x, c_y) :

$$x(s) = c_x + A_x \sin s, \quad y(s) = c_y + A_y \sin s \cos s, \quad s \in [0, 2\pi) \quad (73)$$

where $A_x = \frac{x_{\max} - x_{\min}}{2}$ and $A_y = \frac{y_{\max} - y_{\min}}{2}$ are the half-range amplitudes. The curve crosses itself at (c_x, c_y) at $s = 0$ and $s = \pi$, creating two symmetric loops.

Speed varies around the figure-eight. The parametric speed $\|\dot{\mathbf{p}}\|$ is not uniform: fastest at the crossover, slowest at the tips. For soft SEA springs ($k_s < 200 \text{ N/m}$), high speed at the crossover may exceed the motor bandwidth. Use a lower `--traj-v-max` (e.g. 0.08 m/s) to avoid tracking degradation.

13.3.3 Line Trajectory (Back-and-Forth)

A sinusoidal back-and-forth sweep along y at a fixed x -centre:

$$x(t) = c_x, \quad y(t) = y_{\min} + \frac{y_{\max} - y_{\min}}{2} (1 - \cos(\Omega t)) \quad (74)$$

where $\Omega = 2\pi/T_{\text{lap}}$ and $c_x = \frac{x_{\min} + x_{\max}}{2}$. The EE oscillates smoothly between $y_{\min}$ and $y_{\max}$ at constant $x = c_x$, always arriving at the endpoints with zero velocity.

Recommended --traj-v-max for soft SEA. For $k_s \leq 200$ N/m (AK60-6 torque mode), use `--traj-v-max 0.08` (reduced from the default 0.10) to keep cable extension within the motor bandwidth. At $v_{\max} = 0.15$ m/s the CT controller lags on circle and figure-eight trajectories, producing visible EE deviation at the segments with highest curvature.

14 Simulation Loop (run_simulation)

The `run_simulation()` function builds a complete Drake diagram, initializes state, and runs the simulation.

14.1 Drake Diagram Wiring

14.2 Build Steps

1. **Config:** Create `CupManipulatorTendon` config with joint angles, damping, stiffness
2. **Plant:** Load URDF, weld base, add actuators, add optional `RevoluteSpring`, finalize
3. **Trajectory:** Build rectangle trajectory + move-to-start preamble
4. **Controller:** Add `ComputedTorqueController` with CT gains (K_p , K_d , $\tau_{\max}$)
5. **Actuator:** Add `SEACableActuator` with motor mode, SEA parameters (k_s , b_c , $\tau_{\max}$), and motor config
6. **Wire:** Trajectory $\rightarrow$ CT $\rightarrow$ SEA $\rightarrow$ Plant, plus state feedback to both CT and SEA
7. **Log:** 7 `LogVectorOutput` systems: state and actuation from SEA, diagnostics from SEA, `q_des` from CT, plus 3 trajectory references
8. **Initialize:** Patch zero-mass bodies (Onshape URDF artifacts), set initial $\mathbf{q}$, call `sea.initialize_spring_a` `q2`) ($\delta = 0$)
9. **Run:** Advance in 0.1 s chunks until Ctrl-C; update cable visualization each chunk

14.3 Meshcat Browser Animation

By default, the Meshcat visualizer displays only the *current* robot pose in real time. To enable **replay** (animated playback in the browser after the run), Drake's recording API must be called at the start and end of the simulation:

```

1 # Before the simulation loop
2 if _visualizer is not None:
3     _visualizer.StartRecording()
4
5 try:
```

```

6     # ... simulation loop (Ctrl-C to abort) ...
7     while simulator.get_context().get_time() < duration:
8         simulator.AdvanceTo(...)
9 except KeyboardInterrupt:
10     pass
11 finally:
12     # Always publish, even on early exit
13     if _visualizer is not None:
14         _visualizer.StopRecording()
15         _visualizer.PublishRecording()

```

Listing 13: Meshcat recording: wrap the simulation loop.

Browser playback. After `PublishRecording()`, open `http://localhost:7000` in a browser and click the play button (▶) in the Meshcat animation panel to replay the full simulation from the beginning. Without `StartRecording()`, the browser shows only the final static pose.

`StopRecording()` + `PublishRecording()` **must be called even on early exit.** Place them inside a `finally` block so the animation is published when the user presses Ctrl-C mid-run. Without this, the browser receives no animation data and appears frozen at the initial pose.

14.4 Zero-Mass Body Patching

Onshape-exported URDFs sometimes include bodies with zero mass (no material assigned in CAD). Drake's SAP solver produces NaN forces for zero-mass bodies. The simulation patches these at startup:

```

1 for idx in plant.GetBodyIndices(manipulator.model_instance):
2     body = plant.get_body(idx)
3     if body.default_mass() < 1e-6:
4         body.SetSpatialInertiaInBodyFrame(plant.ctx, _M_PATCH)

```

Listing 14: Zero-mass body patching.

15 Plotting and Diagnostics

The `plot_sea_results()` function produces two figures with comprehensive diagnostics.

15.1 Figure 1: 3×3 Performance Panel

Table 17: Figure 1 layout (3 rows × 3 columns).

Row	Col 0	Col 1	Col 2
0 (EE)	Position: x, y actual vs ref	Velocity: $\dot{x}, \dot{y}$	Acceleration: $\ddot{x}, \ddot{y}$
1 (Joint)	Position: q_1, q_2 [deg]	Velocity: $\dot{q}_1, \dot{q}_2$	Acceleration: $\ddot{q}_1, \ddot{q}_2$
2 (Force)	Torque: τ desired vs applied	Cable tensions: $T_{\text{green}}, T_{\text{red}}$	EE XY path

Joint velocity/acceleration references are reconstructed via $\mathbf{J}^{-1}$:

$$\dot{\mathbf{q}}_{\text{ref}} = \mathbf{J}^{-1}(\mathbf{q}_{\text{des}}) \dot{\mathbf{p}}_{\text{ref}}, \quad \ddot{\mathbf{q}}_{\text{ref}} = \mathbf{J}^{-1}(\mathbf{q}_{\text{des}}) \ddot{\mathbf{p}}_{\text{ref}}$$

15.2 Figure 2: 4×2 SEA Diagnostics

Table 18: Figure 2 layout (5 rows × 2 columns).

Row	Left	Right
0	EE Position X [mm]	EE Position Y [mm]
1	τ_2 desired vs applied (spring lag)	EE XY path overlay
2	Motor cable l_m vs joint side $r_p q_2$	Spring extension δ [mm]
3	Cable tension F [N] (cable can only pull)	Green / Red cable tensions [N]
4	Torque tracking error (shaded, RMS)	EE tracking error $ \mathbf{p}_{\text{act}} - \mathbf{p}_{\text{ref}} $ [mm]

Row 2 is the most informative for understanding SEA dynamics: the **gap** between l_m and $r_p q_2$ is the spring extension δ , which grows with softer springs. Row 3 decomposes the net cable force into antagonistic green (retracting) and red (extending) tensions. Row 4 shows the **torque RMS error** (spring lag) and the resulting **EE tracking error** in mm.

15.3 Comparison Mode

When `--compare` is passed, two simulations run back-to-back:

1. **Spring:** User-specified k_s (default 3.0 N/m)
2. **Rigid:** High k_s ($= 5000$ N/m by default) plus $5\times$ faster motor bandwidth

Both datasets are overlaid on the same plots, showing exactly how spring compliance degrades tracking performance. The EE XY path plot is particularly revealing: the compliant system cuts corners and lags behind the reference.

16 Simulation Results: Stiffness Sweep

This section presents results from four simulations using the **torque-mode** motor model (Cube-Mars AK60-6) with identical CT gains ($K_p = 100$, $K_d = 40$, $\zeta = 2.0$, $\omega_n = 10$ rad/s) and varying spring stiffness $k_s \in \{30, 300, 3000, 30\,000\}$ N/m. All runs track a rectangular EE trajectory.

16.1 Near-Rigid: $k_s = 30\,000\text{ N/m}$

With very high stiffness, the SEA cable behaves nearly identically to a rigid tendon.

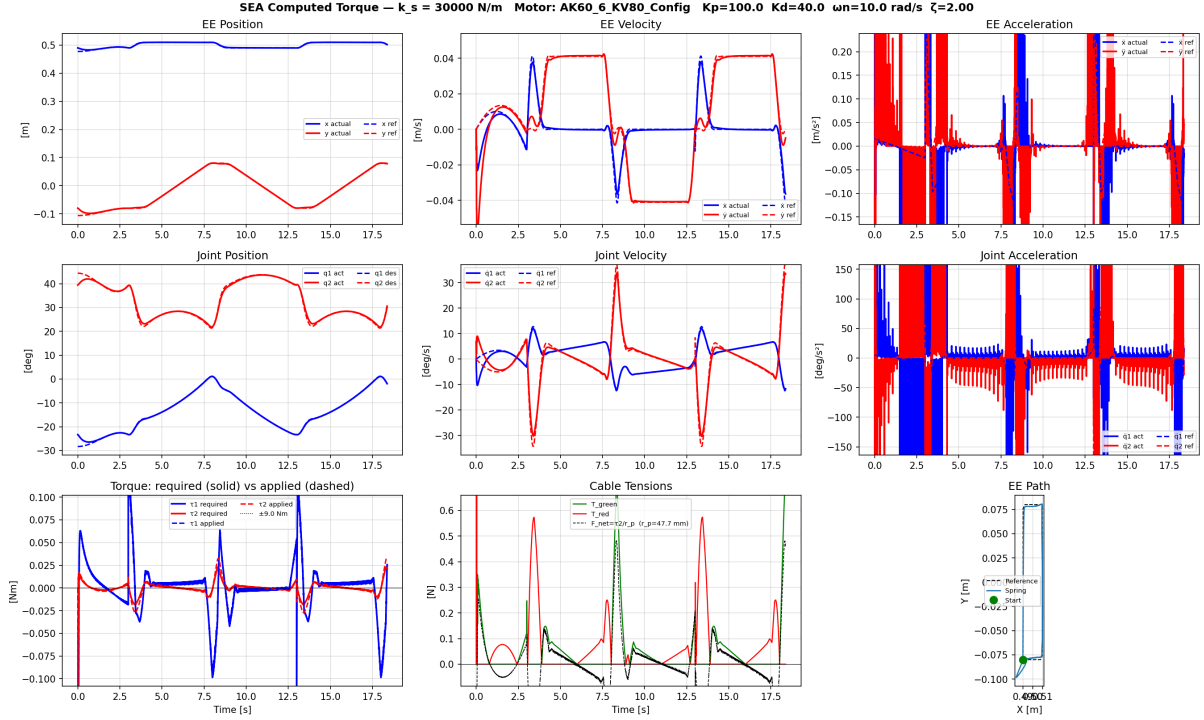


Figure 3: CT 3×3 performance panel at $k_s = 30\,000\text{ N/m}$. EE position (top-left) tracks the rectangular reference closely. Joint positions (middle-left) follow desired values with minimal lag. Applied torque (bottom-left, dashed) almost perfectly overlays desired torque—the spring is stiff enough that $\delta_{ss} = \tau_2/(k_s r_p) \approx 0$ and the SEA acts as a rigid passthrough. Cable tensions alternate between green and red at rectangle corners as the torque direction reverses. The EE path (bottom-right) tightly follows the reference rectangle.

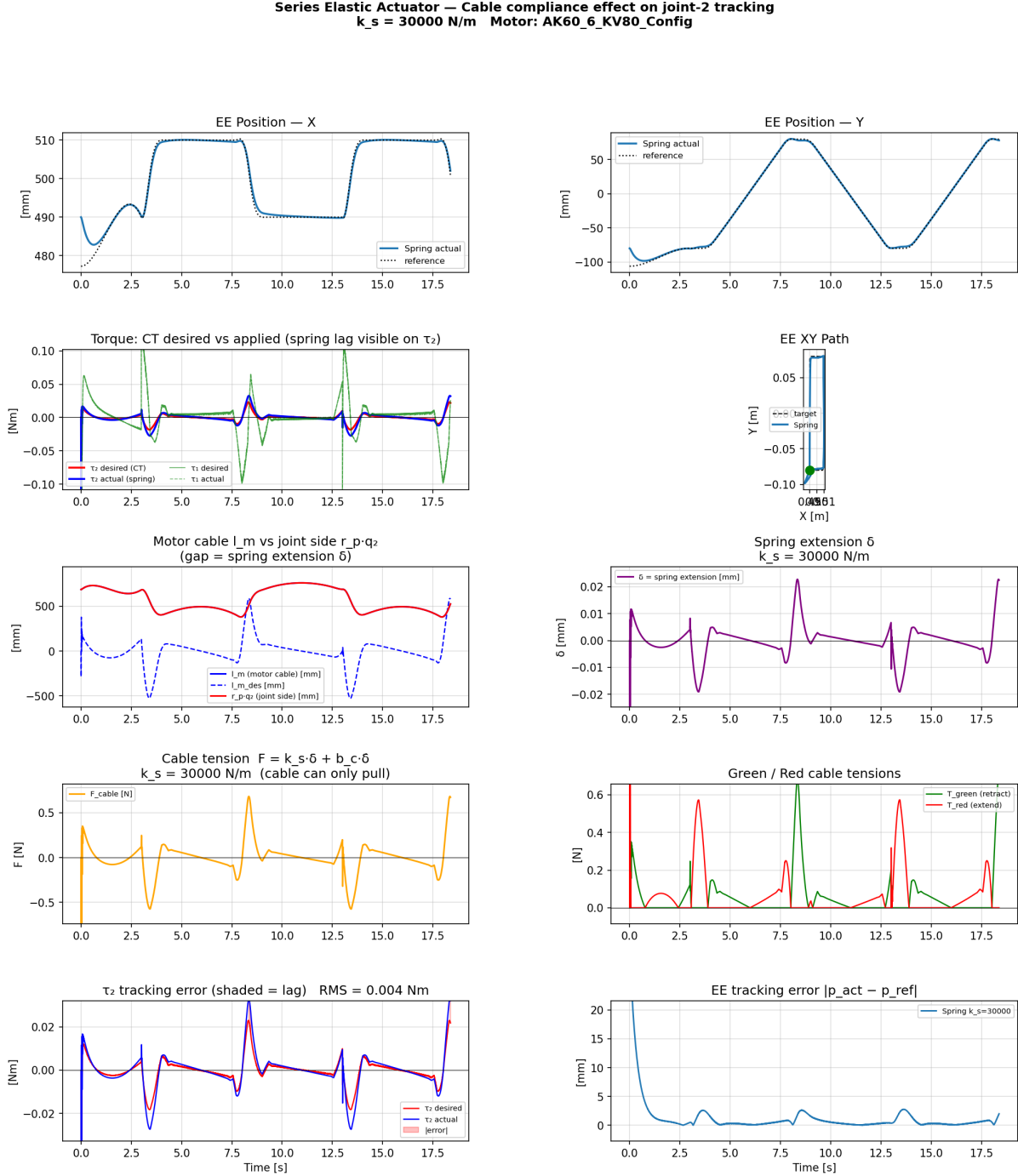


Figure 4: SEA 5×2 diagnostics at $k_s = 30000 \text{ N/m}$. Spring extension δ (row 2, right) peaks at $\pm 0.02 \text{ mm}$ —essentially zero. Motor cable l_m and joint-side $r_p q_2$ (row 2, left) are nearly superimposed, confirming negligible spring deflection. Torque tracking RMS error is only 0.004 Nm . EE tracking error (row 4, right) is dominated by the move-to-start preamble; steady-state tracking stays below 5 mm .

16.2 Moderate Stiffness: $k_s = 3000 \text{ N/m}$

An order of magnitude softer, the spring compliance becomes visible but tracking remains good.

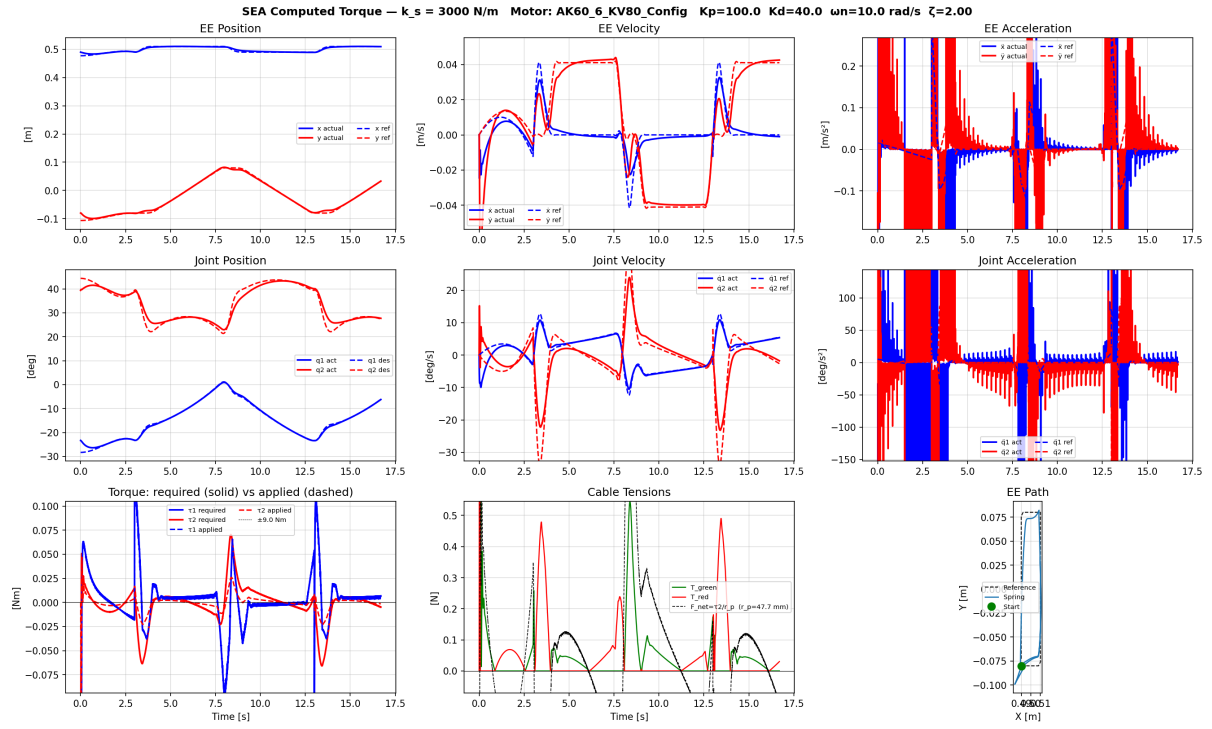


Figure 5: CT 3×3 performance at $k_s = 3000 \text{ N/m}$. EE position tracking remains close to the reference. Velocity and acceleration show slight deviations at rectangle corners. Applied torque τ_2 (bottom-left, dashed) follows desired torque closely but with a visible phase lag at transients—the spring cannot respond instantaneously. Cable tensions peak at $\approx 0.5 \text{ N}$ during the sharp direction changes.

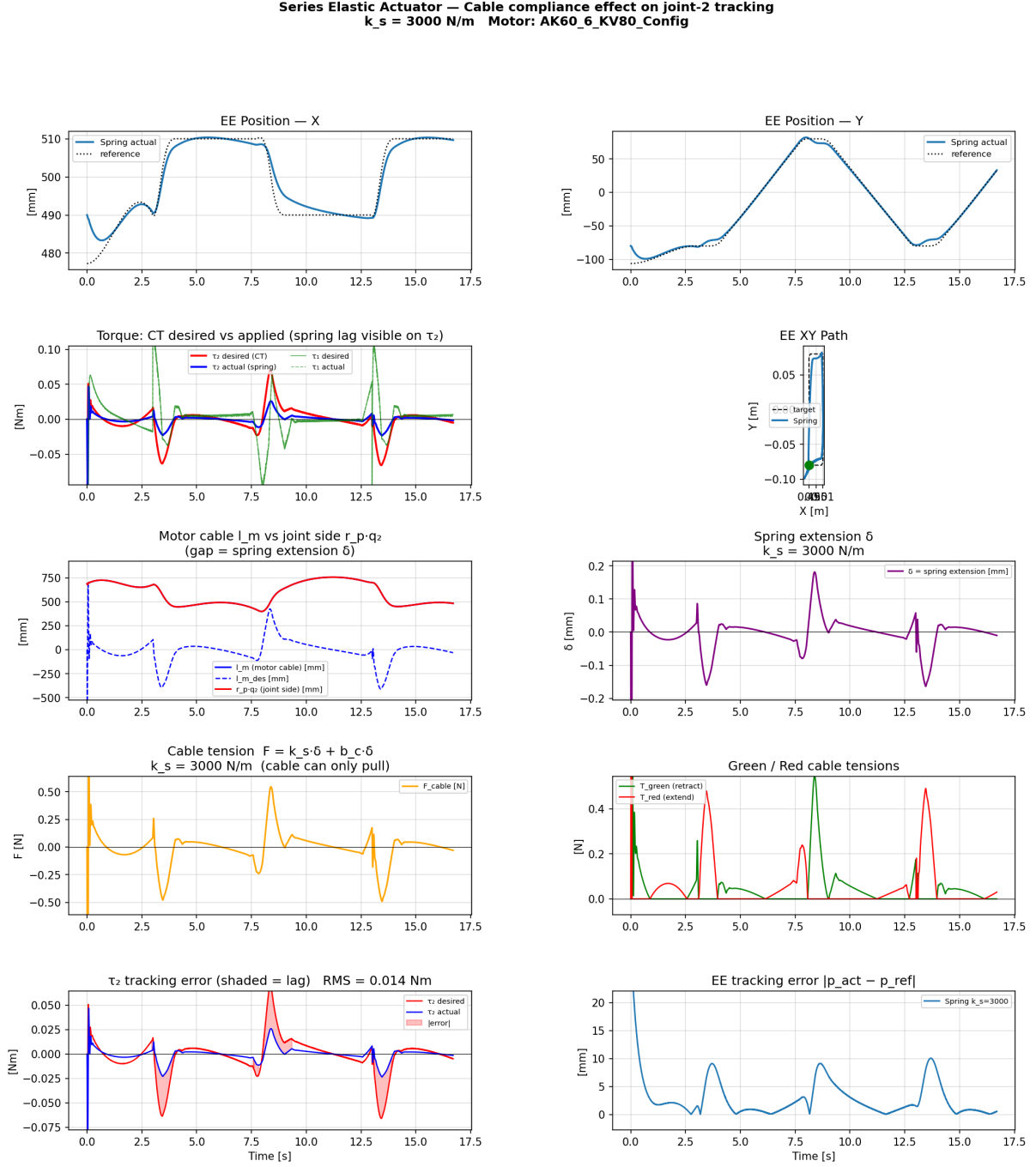


Figure 6: SEA diagnostics at $k_s = 3000 \text{ N/m}$. Spring extension δ (row 2, right) now reaches $\pm 0.2 \text{ mm}$ —an order of magnitude larger than $k_s = 30000$. The gap between l_m and $r_p q_2$ (row 2, left) is visible but small. Torque tracking RMS error is 0.014 Nm , $3.5\times$ worse than the near-rigid case. EE tracking error remains below $\approx 20 \text{ mm}$, with peaks at corners where the spring cannot follow the sudden torque demand.

16.3 Compliant: $k_s = 300 \text{ N/m}$

At this stiffness the spring lag becomes the dominant error source.

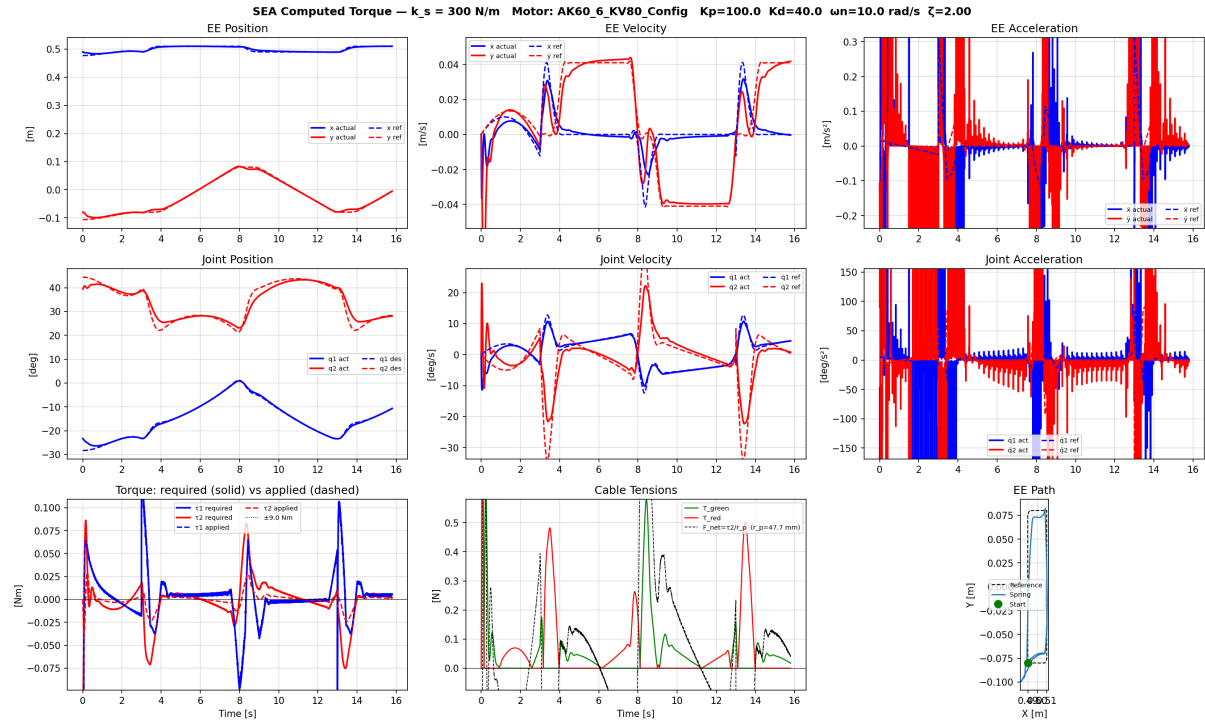


Figure 7: CT 3×3 performance at $k_s = 300$ N/m. EE position shows noticeable deviation from reference at rectangle corners. Joint positions (middle row) show q_2 actual lagging behind desired by several degrees during fast direction changes. Applied τ_2 (bottom-left) now clearly lags the desired torque with visible phase shift. Cable tensions are smaller (peak ≈ 0.4 N) because the spring absorbs more energy before transmitting it.

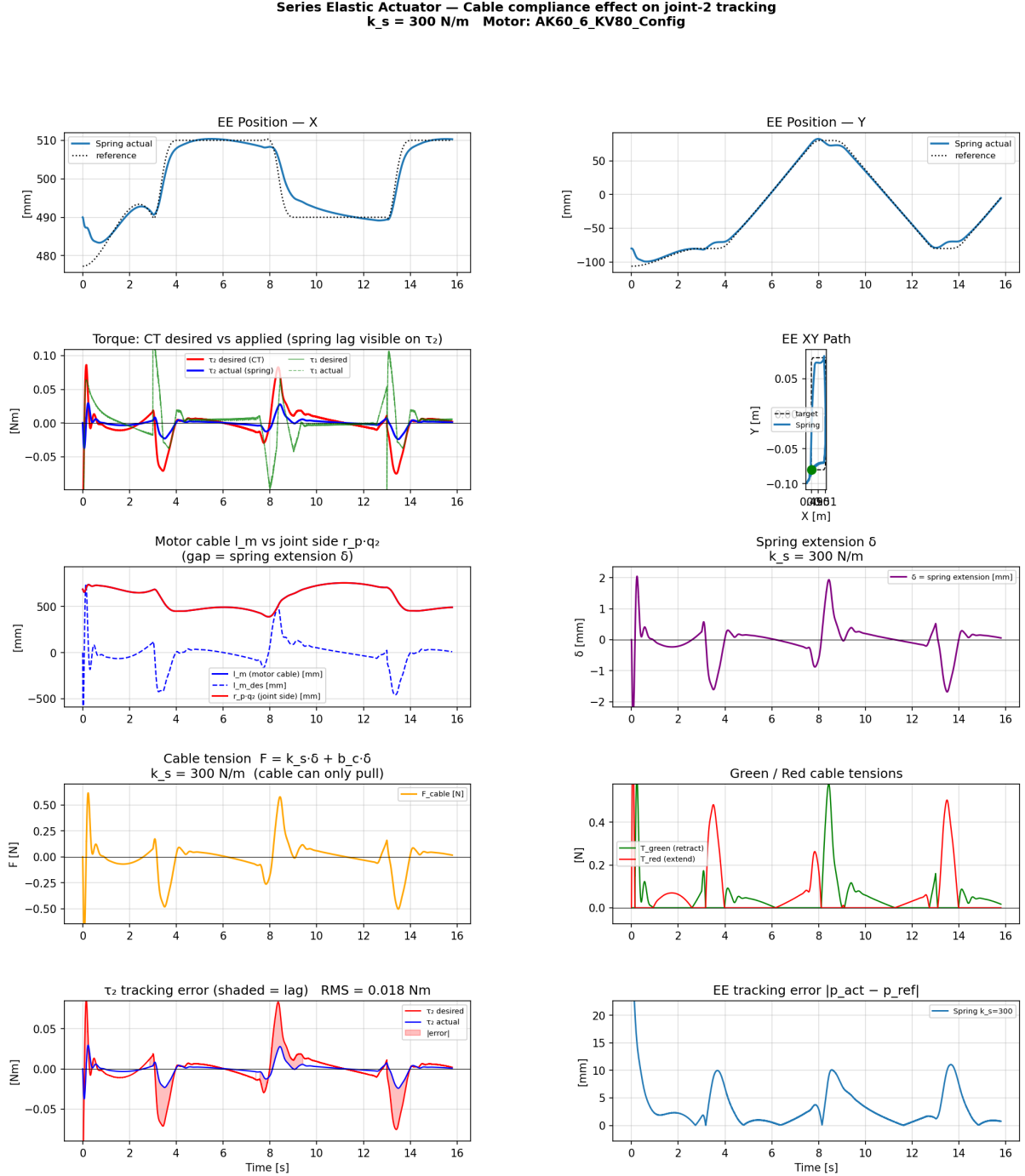


Figure 8: SEA diagnostics at $k_s = 300 \text{ N/m}$. Spring extension δ (row 2, right) reaches $\pm 2 \text{ mm}$ — $10\times$ larger than $k_s = 3000$. The gap between l_m and $r_p q_2$ (row 2, left) is now clearly visible, showing the motor leading the joint. Torque tracking RMS error is 0.018 Nm . EE tracking error peaks at $\approx 20 \text{ mm}$, primarily at corners. In torque mode, reducing k_s by $10\times$ lowers the motor–spring resonance ω_n by $\sqrt{10} \approx 3.2\times$, which slows the motor response and slightly worsens torque tracking ($0.014 \rightarrow 0.018 \text{ Nm}$ RMS). However, the softer spring also reduces force per unit deflection ($F = k_s \delta$), so the overall degradation is moderate.

16.4 Very Soft: $k_s = 30 \text{ N/m}$

At this extreme, the spring dominates the entire system response.

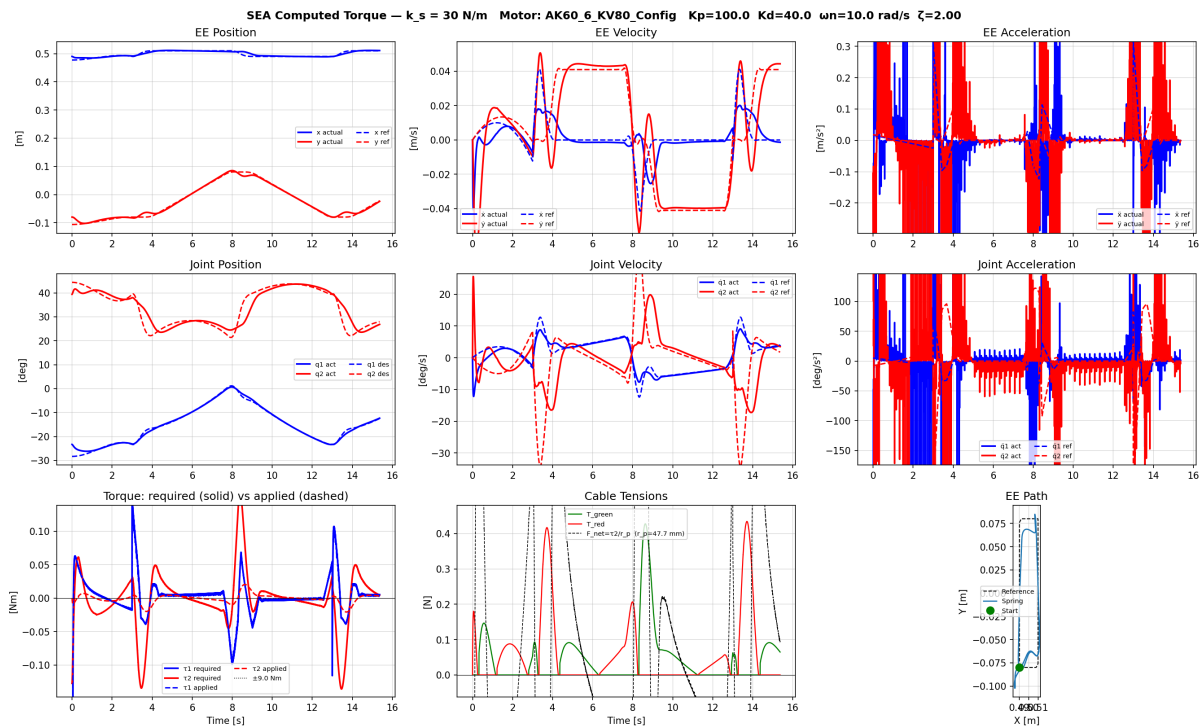


Figure 9: CT 3×3 performance at $k_s = 30$ N/m. EE position (top row) shows significant deviation from reference—the arm overshoots corners and lags on straight segments. Joint-2 position (middle-left, red) deviates substantially from desired (dashed), with the joint settling slowly after each direction change. Applied τ_2 is heavily filtered by the soft spring, with large phase lag. The EE path (bottom-right) departs noticeably from the rectangle, cutting corners and exhibiting hysteresis.

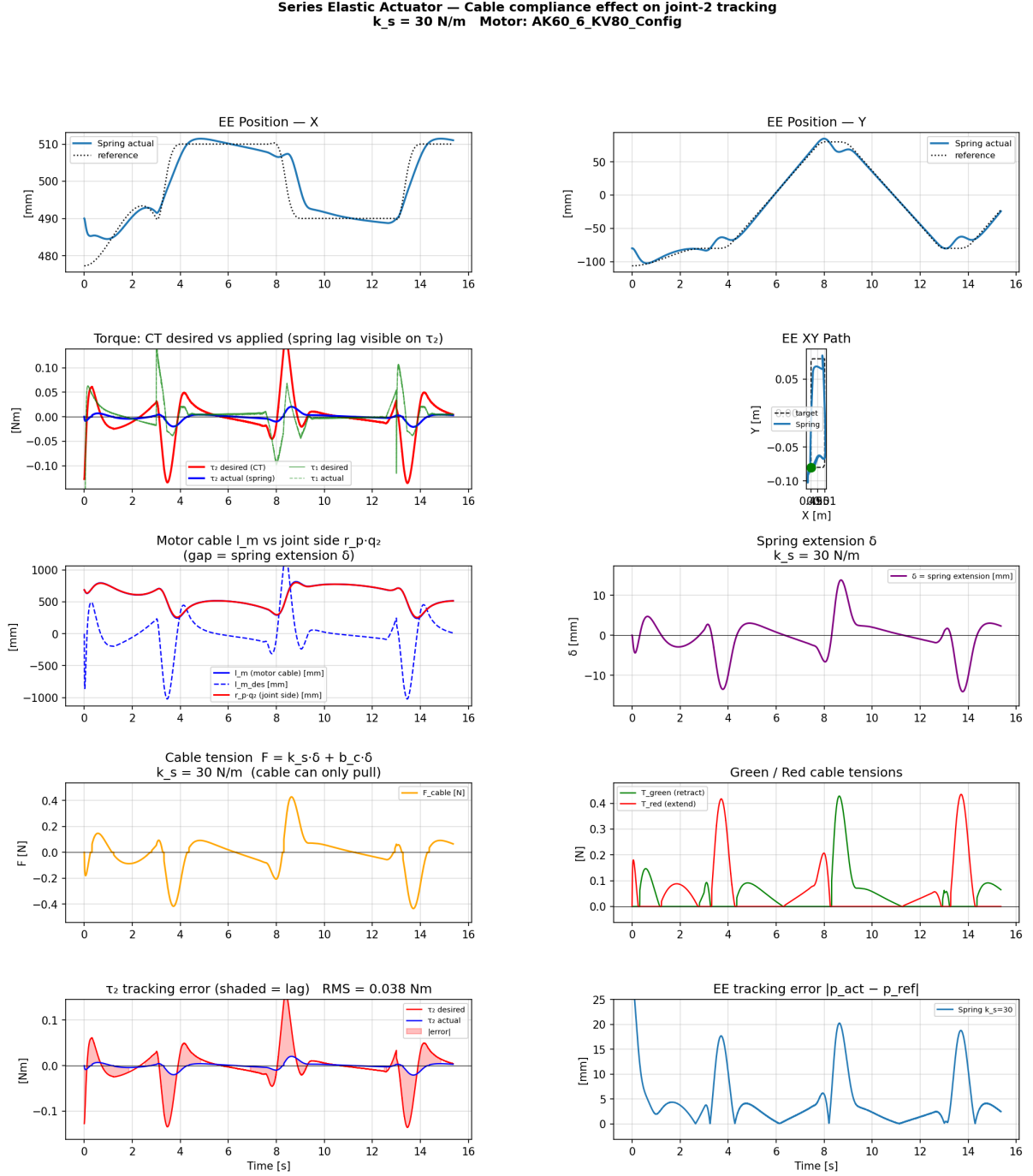


Figure 10: SEA diagnostics at $k_s = 30 \text{ N/m}$. Spring extension δ (row 2, right) reaches $\pm 15 \text{ mm}$ —nearly $1000\times$ the near-rigid case. Motor cable l_m and joint side r_{p-q_2} (row 2, left) diverge dramatically: the motor must travel far ahead of the joint to stretch the soft spring enough to produce the required force. Torque tracking RMS error is 0.038 Nm ($10\times$ worse than near-rigid). EE tracking peaks at $\approx 25 \text{ mm}$. The cable force (row 3) is smaller in magnitude because $F = k_s \delta$ with small k_s , even though δ is large.

16.5 Stiffness Sweep Summary

Table 19: Effect of spring stiffness on tracking quality (torque mode, CubeMars AK60-6).

k_s [N/m]	δ_{peak} [mm]	τ_2 RMS error [Nm]	EE error peak [mm]	Behaviour
30 000	0.02	0.004	< 5	Near-rigid, minimal lag
3 000	0.2	0.014	~ 20	Slight lag at corners
300	2	0.018	~ 20	Visible torque lag, larger δ
30	15	0.038	~ 25	Heavy lag, large motor excursion

Key observations from the stiffness sweep:

1. **Spring extension scales as $1/k_s$:** $\delta_{\text{ss}} = \tau/(k_s r_p)$, so halving k_s doubles the motor excursion needed.
2. **Torque RMS error scales sublinearly with k_s :** Reducing k_s by $1000\times$ (30 000 $\rightarrow$ 30) only increases torque error by $\sim 10\times$, because softer springs also produce less force per unit lag.
3. **EE tracking is dominated by transients:** Steady-state tracking is similar across all stiffnesses; the differences appear at rectangle corners where sudden torque changes expose the spring bandwidth limit.
4. **Motor excursion is the practical constraint:** At $k_s = 30$ N/m, the motor drum must displace ± 15 mm beyond the joint position—approaching the physical travel limits of a real cable drive.

17 Summary of Equations

Complete CT + SEA Pipeline (per timestep):

Computed Torque (inner layer):

$$\text{IK: } \mathbf{q}_{\text{des}} = \text{IK}(\mathbf{p}_{\text{ref}}) \quad (75)$$

$$\text{Diff. IK: } \dot{\mathbf{q}}_{\text{ref}} = \mathbf{J}^{-1} \dot{\mathbf{p}}_{\text{ref}}, \quad \ddot{\mathbf{q}}_{\text{ref}} = \mathbf{J}^{-1} \ddot{\mathbf{p}}_{\text{ref}} \quad (76)$$

$$\text{PD + FF: } \mathbf{a}_{\text{des}} = \ddot{\mathbf{q}}_{\text{ref}} + K_p(\mathbf{q}_{\text{des}} - \mathbf{q}) + K_d(\dot{\mathbf{q}}_{\text{ref}} - \dot{\mathbf{q}}) \quad (77)$$

$$\text{Inv. dynamics: } \boldsymbol{\tau}_{\text{des}} = \mathbf{M}(\mathbf{q}) \mathbf{a}_{\text{des}} + \mathbf{h}(\mathbf{q}, \dot{\mathbf{q}}) \quad (78)$$

SEA Cable Model (Joint 2) — Torque Mode (default):

$$\text{Spring extension: } \delta = r_p \left(\frac{\theta_m}{N} - q_2 \right) \quad (79)$$

$$\text{Cable force: } F = k_s \delta + b_c \dot{\delta} \quad (80)$$

$$\text{Rotor dynamics: } J_m \ddot{\theta}_m = \frac{\tau_{2,\text{des}}}{N} - b_m \dot{\theta}_m - \frac{r_p \cdot F}{N} \quad (81)$$

$$\text{Tensions: } T_{\text{green}} = \max(F, 0), \quad T_{\text{red}} = \max(-F, 0) \quad (82)$$

$$\text{Joint torque: } \tau_2 = r_p \cdot (T_{\text{green}} - T_{\text{red}}) = r_p \cdot F \quad (83)$$

SEA Cable Model (Joint 2) — Position Mode (legacy):

$$\text{Spring extension: } \delta = l_m - r_p q_2 \quad (84)$$

$$\text{Motor servo: } \dot{l}_m = \omega_b (l_m^{\text{des}} - l_m) \quad (85)$$

$$\text{Motor target: } l_m^{\text{des}} = r_p q_2 + \frac{\tau_{2,\text{des}}}{k_s r_p} \quad (86)$$

$$\text{Spring-damper: } F_{\text{raw}} = k_s \delta + b_c (\dot{l}_m - r_p \dot{q}_2) \quad (87)$$

$$\text{Tensions: } T_{\text{green}} = \max(F_{\text{raw}}, 0), \quad T_{\text{red}} = \max(-F_{\text{raw}}, 0) \quad (88)$$

$$\text{Joint torque: } \tau_2 = r_p \cdot (T_{\text{green}} - T_{\text{red}}) = r_p \cdot F_{\text{raw}} \quad (89)$$

Joint 1: $\tau_1 = \tau_{1,\text{des}}$ (rigid direct drive, unchanged from CT)

Visualization: Spring coil length = $L_{\text{rest}} + |\delta|$ (proportional to tension)

